



OpenShift Application Assessment

ShiftWise AI

ShiftWise AI

Outubro de 2026



Termos de Uso e Responsabilidade

Este relatório é produzido automaticamente pelo ShiftWise AI, um projeto experimental e de código aberto. Seu conteúdo pode conter erros, omissões, inconsistências ou informações desatualizadas e não constitui recomendação profissional, jurídica, técnica, financeira ou de segurança.

A ShiftWise AI, seus mantenedores, contribuidores e distribuidores não garantem a exatidão, integridade, adequação ou disponibilidade das informações geradas. Na máxima extensão permitida pela legislação aplicável, não se responsabilizam por decisões, perdas, danos ou consequências decorrentes do uso, da interpretação ou da impossibilidade de uso deste relatório.

O usuário é exclusivamente responsável por revisar, validar e aprovar todas as informações antes de utilizá-las em decisões, configurações ou ambientes. Qualquer uso ou implementação ocorre por conta e risco do usuário e deve ser submetido à avaliação de profissionais qualificados quando aplicável.

Ao utilizar este relatório, o usuário declara estar ciente e de acordo com estas condições.



Histórico de versionamento

Versão	Data	Contribuição	Função	Descrição
1.0		ShiftWise AI		



Participantes do Projeto

ShiftWise AI

Nome	Cargo/Função	Endereço de e-mail
------	--------------	--------------------

Cliente

Nome	Cargo/Função	Endereço de e-mail
------	--------------	--------------------



Sumário

Termos de Uso e Responsabilidade	1
Histórico de versionamento	2
Participantes do Projeto	3
ShiftWise AI	3
Cliente	3
1. Legenda de siglas	8
Categorias de findings	8
Níveis de severidade	9
Níveis de confiança	9
Termos técnicos	9
2. Sumário executivo	10
3. Escopo da análise	11
4. Fontes de dados	12
5. Visão geral do namespace	14
Comparativo request / limit / uso (snapshot)	14
Arquitetura	14
Visualizações	16
Requests do namespace × capacidade allocatable do pool	16
Distribuição de QoS	17
6. Workloads identificados	19
Inventário — Services	19
Inventário — Routes (exposição externa)	20
Inventário — ConfigMaps	20
Inventário — Secrets referenciados	24
Inventário — Logs de pods (amostra)	24
RES-CONFIG-001 — [CONFIG] (42 workloads; RES-CONFIG-001 ... RES-CONFIG-042)	25
Workloads consolidados (todos os controllers)	31
Detalhamento por workload	32
Placement observado	34
Comunicação	34
1. Externa → OpenShift	34
Comunicação externa — kubeoptix-dashboard	34
2. Comunicação interna	35
Comunicação interna — analyzer-api	36
Comunicação interna — core-ai	37
Comunicação interna — core-ai-api	38
Comunicação interna — harvester-api	39



Comunicação interna — kubeoptix-dashboard	40
Comunicação interna — reporter	41
Comunicação interna — reporter-api	42
3. Dependências externas	43
Dependências externas — kubeoptix-analyzer	43
Dependências externas — kubeoptix-core-ai	45
Dependências externas — kubeoptix-dashboard	47
Dependências externas — kubeoptix-harvester	49
Dependências externas — kubeoptix-reporter	51
Visualizações	53
Diagrama de workloads — kubeoptix-analyzer	53
Diagrama de workloads — kubeoptix-core-ai	54
Diagrama de workloads — kubeoptix-dashboard	56
Diagrama de workloads — kubeoptix-harvester	58
Diagrama de workloads — kubeoptix-reporter	60
Workloads por app group	62
7. Análise de CPU	64
Configuração (request/limit)	64
Uso real (PodMetrics)	64
Visualizações	64
CPU request por workload (por pod)	64
CPU limit por workload (por pod)	65
CPU request × limit por workload (por pod)	66
RES-CPU-001	67
8. Análise de memória	68
Visualizações	68
Memória request por workload (por pod)	68
Memória limit por workload (por pod)	69
Memória request × limit por workload (por pod)	69
RES-MEM-001	70
9. Análise de QoS	71
Visualizações	71
Distribuição de QoS	71
10. Análise de réplicas	72
RES-REPLICA-001 — [REPLICA] (5 workloads; RES-REPLICA-001 ... RES-REPLICA-005)	72
RES-REPLICA-006	73
11. Análise de probes	74
RES-PROBE-001	74
RES-PROBE-002	74



RES-PROBE-003 — [PROBE] (2 workloads; RES-PROBE-003 ... RES-PROBE-004)	75
12. Análise de scheduling	76
13. Análise de storage	77
RES-STORAGE-001 — [STORAGE] (6 workloads; RES-STORAGE-001 ... RES-STORAGE-006)	77
RES-STORAGE-007	78
14. Correlação workload × worknode	79
Visualizações	79
Placement workload × worknode	79
RES-WNODE-001	80
RES-WNODE-002	81
15. Anomalias identificadas	82
ML-MLANOM-001	82
ML-MLCLUST-001	82
ML-MLCOMP-001	83
ML-MLCOMP-002	83
ML-MLSIM-001	84
ML-MLSIM-002	84
ML-MLSIM-003	85
16. Findings	86
Visualizações	90
Findings por severidade	90
Findings por categoria	91
17. Oportunidades de otimização	93
18. Riscos	94
19. Recomendações	95
Plano de ação	95
Lista consolidada	96
20. Conclusão	97
21. Limitações da análise	98
22. Referências	99
Documentação técnica — Red Hat	99
Documentação técnica complementar (upstream)	99
Referências bibliográficas — Red Hat	100
Referências bibliográficas complementares	100



Namespace analisado: shiftwise-ai **Fonte dos dados:** /app/data/assessment/shiftwise-ai **Worknodes:** /app/data/assessment/worknodes **Data de geração:** 20/08/2026 18:10 UTC **Gerado por:** kubeoptix-core-ai

Este relatório foi produzido exclusivamente a partir dos artefatos YAML ingeridos pelo agente. Nenhum dado foi inventado; onde a evidência é insuficiente, isso é indicado explicitamente. **Request/limit** referem-se a configuração declarada; **usage** refere-se a métricas de runtime (PodMetrics), quando disponíveis.



1. Legenda de siglas

Glossário das siglas e abreviações utilizadas neste relatório.

Categorias de findings

Sigla	Descrição
CPU	Dimensionamento e uso de CPU
MEM	Dimensionamento e uso de memória
RES	Recursos declarados (request/limit gerais)
QOS	Quality of Service — classes Guaranteed, Burstable e BestEffort
REPLICA	Réplicas, HPA e disponibilidade
PROBE	Liveness, readiness e startup probes
SCHED	Scheduling, affinity e distribuição de pods
STORAGE	Volumes persistentes e PVCs
WNODE	Correlação workload × worknode
ROUTE	Routes e exposição HTTP/TLS
CONFIG	ConfigMaps e configuração
LOG	Logs coletados dos pods
OPER	Operadores (OLM/CSV)
MLSTAT	Sinais estatísticos locais
MLCOMP	Comparação estatística entre workloads
MLANOM	Deteção de anomalias estatísticas
MLCLUST	Agrupamento (clustering) de workloads
MLSIM	Similaridade entre workloads



Níveis de severidade

Sigla	Descrição
CRITICAL	Risco imediato ou impacto operacional grave
HIGH	Problema significativo que requer atenção prioritária
MEDIUM	Desvio relevante; planejar correção
LOW	Oportunidade de melhoria ou risco residual baixo
INFO	Informativo; sem ação obrigatória

Níveis de confiança

Sigla	Descrição
HIGH	Evidência direta nos artefatos coletados
MEDIUM	Inferência com suporte parcial nos dados
LOW	Hipótese com evidência limitada

Termos técnicos

Sigla	Descrição
HPA	Horizontal Pod Autoscaler
QoS	Quality of Service — classe de garantia de recursos do pod
PVC	PersistentVolumeClaim
TLS	Transport Layer Security
RES-*	Prefixo dos IDs de findings determinísticos (ex.: RES-CPU-001)
ML-*	Prefixo dos IDs de findings da camada estatística (ex.: ML-MLANOM-001)
PodMetrics	Métricas de runtime coletadas via API metrics.k8s.io



2. Sumário executivo

O namespace **shiftwise-ai** foi analisado com **5 workload(s)** e **1 nó(s) worker** no dump coletado. Foram registrados **70 findings** (0 CRITICAL, 0 HIGH, 8 MEDIUM).

Principais achados:

4. **Scheduling e resiliência:** concentração de réplicas ou placement fora do pool esperado — avaliar nodeSelector, podAntiAffinity e topologySpreadConstraints.
5. **Sinais ML locais:** 7 indício(s) estatístico(s) complementar(es) — interpretar como hipóteses para investigação, não como defeitos confirmados.

Workload	Kind	Réplicas (desej./prontas)	QoS
kubeoptix-analyzer	StatefulSet	1/1	Burstable
kubeoptix-core-ai	StatefulSet	1/1	Burstable
kubeoptix-dashboard	StatefulSet	1/1	Burstable
kubeoptix-harvester	StatefulSet	1/1	Burstable
kubeoptix-reporter	StatefulSet	1/1	Burstable



3. Escopo da análise

- **Workloads analisados:** 5
 - **Worknodes considerados:** 1
 - **Arquivos encontrados:** 1010
 - **Arquivos processados:** 98
 - **Arquivos ignorados:** 855
 - **Erros de parsing:** 0
 - **Camada ML local:** ativada
-



4. Fontes de dados

Tipos de artefato considerados nesta execução:

- Workloads: Deployment, StatefulSet, DaemonSet, DeploymentConfig, Job, CronJob, ReplicationController (`apps/{app}/` e `resources/`)
- Pods (`apps/*/pods/`, `resources/pods/`)
- Autoscaling: HPA, VPA (`apps/*/hpa|vpa/` ou `resources/`)
- PDB (`apps/*/pdb/` ou `resources/poddisruptionbudgets.policy/`)
- PodMetrics (`apps/*/pods.metrics.k8s.io/`, `resources/pods.metrics.k8s.io/`)
- PVCs (`apps/*/persistentvolumeclaims/`, `resources/persistentvolumeclaims/`)
- Services (`resources/services/`, `apps/*/services/`)
- Routes (`resources/routes.route.openshift.io/`, `apps/*/routes/`)
- ConfigMaps (`resources/configmaps/`, `apps/*/configmaps/`)
- Secrets (referências por nome nos workloads)
- ClusterServiceVersions / PackageManifests (OLM)
- Logs de pods (`apps/*/pod-logs/*.log`)
- Worknodes (`worknodes/*.yaml`)

Correlação: pods são associados ao controller canônico via ownerReferences (Pod → ReplicaSet → Deployment, Pod → StatefulSet, Job → CronJob, etc.). A pasta `apps/{app_group}/` é varrida recursivamente e fundida com `resources/`. ReplicaSets históricos não geram workloads duplicados.

Não analisados nesta versão: conteúdo de Secrets (apenas referências por nome), eventos, NetworkPolicies, séries temporais.

Arquivos processados (amostra):

- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-analyzer-1-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-core-ai-10-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-core-ai-6-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-core-ai-7-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-core-ai-8-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-core-ai-9-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-dashboard-3-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-dashboard-4-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-dashboard-5-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-dashboard-6-build.log`
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-dashboard-7-build.log`



-
- `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-harvester-1-build.log`
 - `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-harvester-2-build.log`
 - `shiftwise-ai/apps/__no_app__/pod-logs/kubeoptix-reporter-1-build.log`
 - `shiftwise-ai/apps/kubeoptix-analyzer/pod-logs/kubeoptix-analyzer-0.log`
 - `shiftwise-ai/apps/kubeoptix-core-ai/pod-logs/kubeoptix-core-ai-0.log`
 - `shiftwise-ai/apps/kubeoptix-dashboard/pod-logs/kubeoptix-dashboard-0.log`
 - `shiftwise-ai/apps/kubeoptix-harvester/pod-logs/kubeoptix-harvester-0.log`
 - `shiftwise-ai/apps/kubeoptix-reporter/pod-logs/kubeoptix-reporter-0.log`
 - `shiftwise-ai/resources/configmaps/kube-root-ca.crt.yaml`
 - ... e mais 78 arquivo(s)
-



5. Visão geral do namespace

Métrica	Valor	Observação
CPU request (soma namespace)	1800m	—
CPU limit (soma namespace)	7500m	—
Memória request (soma namespace)	3648Mi	—
Memória limit (soma namespace)	14592Mi	—
Nós worker (coletados)	1	Total no dump de worknodes
Pool de scheduling relevante	1 nó(s)	todos os nós coletados
CPU allocatable (pool relevante)	7800m	—
Memória allocatable (pool relevante)	23571.82Mi	—

Comparativo request / limit / uso (snapshot)

Uso real é **snapshot pontual** (PodMetrics); requests/limits são configuração declarada nos controllers de workload.

Recurso	Total request	Total limit	Uso real (snapshot)	Uso vs request	Uso vs limit
CPU	1800m	7500m	indisponível	—	—
Memória	3648Mi	14592Mi	indisponível	—	—

Arquitetura

Diagrama gerado com **KubeDiagrams** a partir dos manifests YAML do namespace. Relacionamentos são apenas os comprováveis no inventário (selector, **spec.to**, volumes, envFrom, scaleTargetRef).

Legenda de cores (agrupamentos):

- **Workloads** — fundo azul (**Deployment**, **StatefulSet**, **DaemonSet**, **DeploymentConfig**, **Job**, **CronJob**, **HPA/VPA/PDB**).
- **Pods** — fundo azul-claro; pods equivalentes aparecem **uma vez**, com o rótulo **nome (N replicas)**.
- **Networking** — fundo verde (**Service**, **Route**, **Ingress**, **NetworkPolicy**); Services exibem a porta



(nome:8000).

- **Storage** — fundo âmbar (PVC, PV, StorageClass).
- **Configuration** — fundo cinza (ConfigMap, Secret, ServiceAccount).

Leitura sugerida: à esquerda Workloads → Configuration → Storage; à direita Networking.

Tipos de aresta: `selector` (tracejada), referência direta (sólida), `controller/owner` (pontilhada).

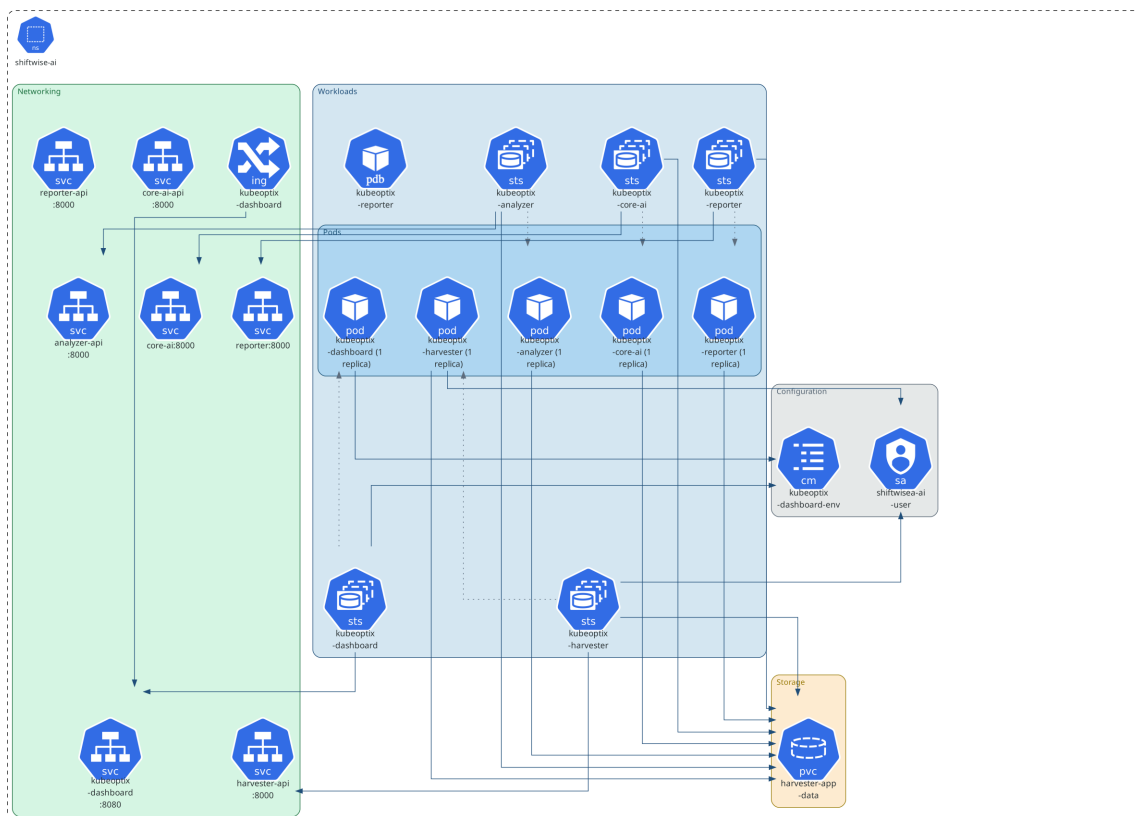
Artefatos de Build (BuildConfig, Build, ImageStream, pods `*-build`), artefatos padrão de plataforma (SA `default/builder/deployer`, ConfigMaps `*-ca`, tokens de SA) e recursos derivados (Endpoints, Leases, ReplicaSets intermediários) são omitidos para legibilidade.

Ajustes por ambiente: `KUBEOPTIX_DIAGRAM_INCLUDE_COMMON_OCP`, `KUBEOPTIX_DIAGRAM_INCLUDE_PATH_REGEX` e `KUBEOPTIX_DIAGRAM_EXCLUDE_PATH_REGEX`.

Conteúdo do namespace no diagrama: Workloads: 5 | Pods: 5 | Networking: 8 | Storage: 1 | Config: 2 | Autoscalers/PDB: 1

Manifests YAML utilizados:

- `resources/configmaps/kubeoptix-dashboard-env.yaml`
- `resources/serviceaccounts/shiftwisea-ai-user.yaml`
- `resources/statefulsets.apps/kubeoptix-analyzer.yaml`
- `resources/statefulsets.apps/kubeoptix-core-ai.yaml`
- `resources/statefulsets.apps/kubeoptix-dashboard.yaml`
- `resources/statefulsets.apps/kubeoptix-harvester.yaml`
- `resources/statefulsets.apps/kubeoptix-reporter.yaml`
- `resources/poddisruptionbudgets.policy/kubeoptix-reporter.yaml`
- `resources/pods/kubeoptix-analyzer-0.yaml`
- `resources/pods/kubeoptix-core-ai-0.yaml`
- `resources/pods/kubeoptix-dashboard-0.yaml`
- `resources/pods/kubeoptix-harvester-0.yaml`
- ... e mais 10 arquivo(s)

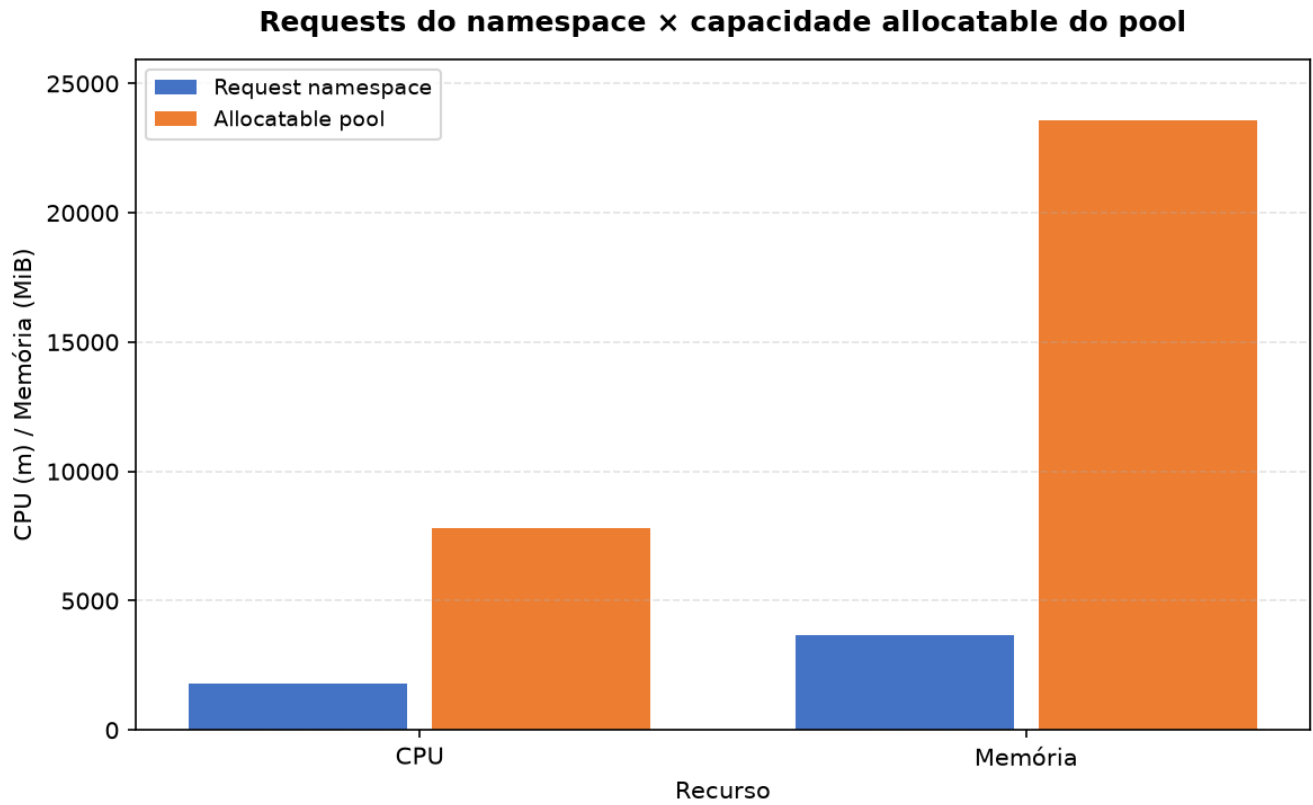


Visualizações

Requests do namespace × capacidade allocatable do pool

Os requests agregados do namespace cabem no pool de scheduling relevante?

Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: Request namespace, Allocatable pool. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



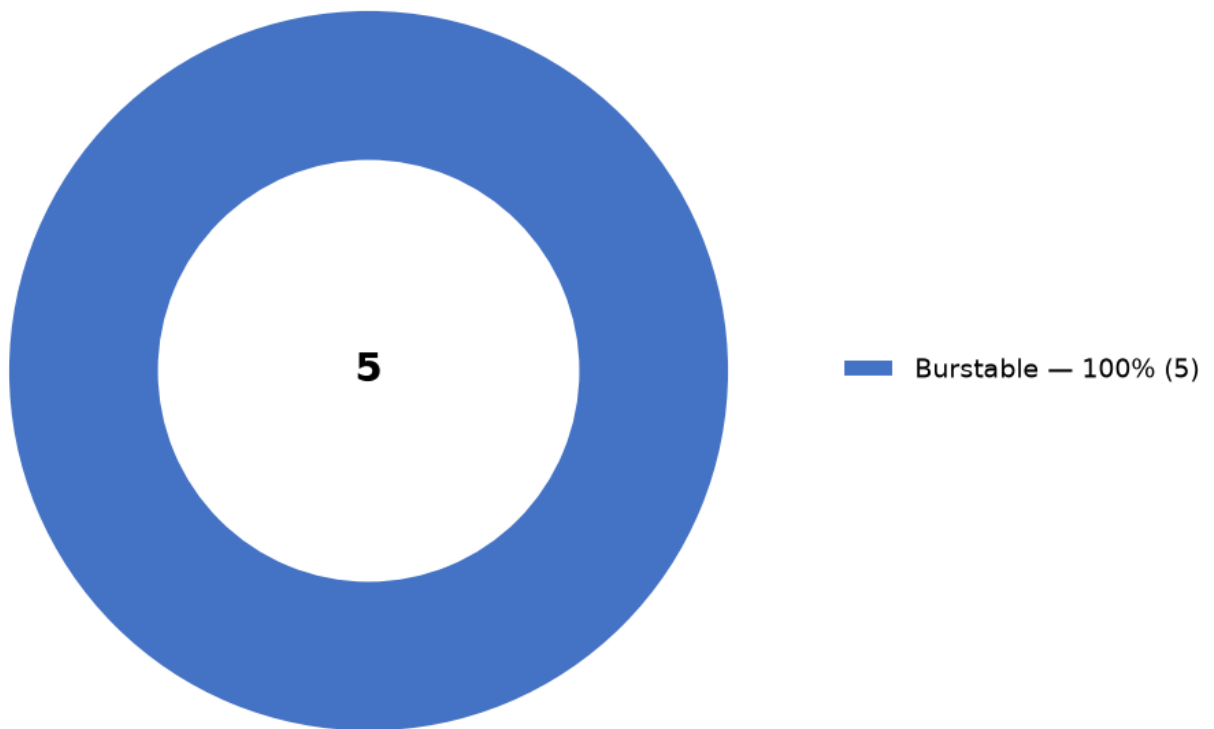
Distribuição de QoS

Como os workloads se distribuem entre classes QoS?

Valor calculado: distribuição entre 5 unidade(s) — Burstable: 5. Contagens derivadas dos dados ingeridos, sem inferência adicional.



Distribuição de QoS





6. Workloads identificados

Inventário — Services

Service	Tipo	ClusterIP	Porta → TargetPort	Seletores
analyzer-api	ClusterIP	10.217.5.192	8000 → 8000	app.kubernetes.io/instance=kubeoptix-analyzer, app.kubernetes.io/name=kubeoptix-analyzer
core-ai	ClusterIP	None	8000 → http	app.kubernetes.io/instance=kubeoptix-core-ai, app.kubernetes.io/name=kubeoptix-core-ai
core-ai-api	ClusterIP	10.217.5.158	8000 → http	app.kubernetes.io/instance=kubeoptix-core-ai, app.kubernetes.io/name=kubeoptix-core-ai
harvester-api	ClusterIP	10.217.4.119	8000 → 8000	app.kubernetes.io/instance=kubeoptix-harvester, app.kubernetes.io/name=kubeoptix-harvester
kubeoptix-dashboard	ClusterIP	10.217.5.92	8080 → http	app.kubernetes.io/name=kubeoptix-dashboard
reporter	ClusterIP	None	8000 → http	app.kubernetes.io/instance=kubeoptix-reporter, app.kubernetes.io/name=kubeoptix-reporter



Service	Tipo	ClusterIP	Porta → TargetPort	Seletor
reporter-api	ClusterIP	10.217.4.211	8000 → 8000	app.kubernetes.io/instance=kubeoptix-reporter, app.kubernetes.io/name=kubeoptix-reporter

Inventário — Routes (exposição externa)

Route	Host	Serviço destino	Porta	Termination	Política insegura
kubeoptix-dashboard	kubeoptix-dashboard-shiftwise-ai.apps-crc.testing	kubeoptix-dashboard	http	edge	Redirect

Inventário — ConfigMaps

ConfigMap	Chaves	Consumido por workload?	App group
kube-root-ca.crt	ca.crt	Não	—
kubeoptix-analyzer-1-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-analyzer-1-global-ca	ca-bundle.crt	Não	—
kubeoptix-analyzer-1-sys-config	—	Não	—
kubeoptix-core-ai-10-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-core-ai-10-global-ca	ca-bundle.crt	Não	—



ConfigMap	Chaves	Consumido por workload?	App group
kubeoptix-core-ai-10-sys-config	—	Não	—
kubeoptix-core-ai-6-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-core-ai-6-global-ca	ca-bundle.crt	Não	—
kubeoptix-core-ai-6-sys-config	—	Não	—
kubeoptix-core-ai-7-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-core-ai-7-global-ca	ca-bundle.crt	Não	—
kubeoptix-core-ai-7-sys-config	—	Não	—
kubeoptix-core-ai-8-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-core-ai-8-global-ca	ca-bundle.crt	Não	—
kubeoptix-core-ai-8-sys-config	—	Não	—
kubeoptix-core-ai-9-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-core-ai-9-global-ca	ca-bundle.crt	Não	—
kubeoptix-core-ai-9-sys-config	—	Não	—



ConfigMap	Chaves	Consumido por workload?	App group
kubeoptix-dashboard-3-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-dashboard-3-global-ca	ca-bundle.crt	Não	—
kubeoptix-dashboard-3-sys-config	—	Não	—
kubeoptix-dashboard-4-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-dashboard-4-global-ca	ca-bundle.crt	Não	—
kubeoptix-dashboard-4-sys-config	—	Não	—
kubeoptix-dashboard-5-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-dashboard-5-global-ca	ca-bundle.crt	Não	—
kubeoptix-dashboard-5-sys-config	—	Não	—
kubeoptix-dashboard-6-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-dashboard-6-global-ca	ca-bundle.crt	Não	—
kubeoptix-dashboard-6-sys-config	—	Não	—
kubeoptix-dashboard-7-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—



ConfigMap	Chaves	Consumido por workload?	App group
kubeoptix-dashboard-7-global-ca	ca-bundle.crt	Não	—
kubeoptix-dashboard-7-sys-config	—	Não	—
kubeoptix-dashboard-env	ANALYZER_API_URL, CORE_AI_API_URL, ENV, HARVESTER_API_URL, REPORTER_API_URL, TZ	Sim	—
kubeoptix-harvester-1-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-harvester-1-global-ca	ca-bundle.crt	Não	—
kubeoptix-harvester-1-sys-config	—	Não	—
kubeoptix-harvester-2-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-harvester-2-global-ca	ca-bundle.crt	Não	—
kubeoptix-harvester-2-sys-config	—	Não	—
kubeoptix-reporter-1-ca	default-route-openshift-image-registry.apps-crc.testing, service-ca.crt	Não	—
kubeoptix-reporter-1-global-ca	ca-bundle.crt	Não	—
kubeoptix-reporter-1-sys-config	—	Não	—
openshift-service-ca.crt	service-ca.crt	Não	—



Inventário — Secrets referenciados

Nenhum conteúdo de Secret foi lido ou reproduzido — apenas nomes e forma de uso.

Secret (nome)	Forma de uso	Workload
llm	envFrom/volume	kubeoptix-analyzer

Inventário — Logs de pods (amostra)

App	Pod	Linhas	Vazio?	Níveis detectados
kubeoptix-analyzer	kubeoptix-analyzer-0	93	Não	—
__no_app__	kubeoptix-analyzer-1-build	1	Não	—
kubeoptix-core-ai	kubeoptix-core-ai-0	6	Não	INFO:6
__no_app__	kubeoptix-core-ai-10-build	294	Não	ERROR:1
__no_app__	kubeoptix-core-ai-6-build	1	Não	—
__no_app__	kubeoptix-core-ai-7-build	1	Não	—
__no_app__	kubeoptix-core-ai-8-build	1	Não	—
__no_app__	kubeoptix-core-ai-9-build	296	Não	ERROR:1
kubeoptix-dashboard	kubeoptix-dashboard-0	5	Não	—
__no_app__	kubeoptix-dashboard-3-build	263	Não	—
__no_app__	kubeoptix-dashboard-4-build	263	Não	—
__no_app__	kubeoptix-dashboard-5-build	263	Não	—
__no_app__	kubeoptix-dashboard-6-build	263	Não	—



App	Pod	Linhas	Vazio?	Níveis detectados
__no_app__	kubeoptix-dashboard-7-build	260	Não	—
kubeoptix-harvester	kubeoptix-harvester-0	300	Não	INFO:300
__no_app__	kubeoptix-harvester-1-build	1	Não	—
__no_app__	kubeoptix-harvester-2-build	1	Não	—
kubeoptix-reporter	kubeoptix-reporter-0	300	Não	INFO:300
__no_app__	kubeoptix-reporter-1-build	1	Não	—

Findings de inventário / observabilidade:

RES-CONFIG-001 — [CONFIG] (42 workloads; RES-CONFIG-001 ... RES-CONFIG-042)

Severidade: INFO Confiança: HIGH Workloads afetados (42): —

Evidências:

Workload	Container	Evidência
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']
—	—	ConfigMap sem referência em workloads: keys=['ca-bundle.crt']
—	—	ConfigMap sem referência em workloads: keys=[]
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']



Workload	Container	Evidência
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>



Workload	Container	Evidência
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']
—	—	ConfigMap sem referência em workloads: keys=['ca-bundle.crt']
—	—	ConfigMap sem referência em workloads: keys=[]
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']
—	—	ConfigMap sem referência em workloads: keys=['ca-bundle.crt']
—	—	ConfigMap sem referência em workloads: keys=[]
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']
—	—	ConfigMap sem referência em workloads: keys=['ca-bundle.crt']
—	—	ConfigMap sem referência em workloads: keys=[]
—	—	ConfigMap sem referência em workloads: keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']
—	—	ConfigMap sem referência em workloads: keys=['ca-bundle.crt']



Workload	Container	Evidência
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>



Workload	Container	Evidência
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['default-route-openshift-image-registry.apps-crc.testing', 'service-ca.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=['ca-bundle.crt']</code>
—	—	ConfigMap sem referência em workloads: <code>keys=[]</code>

Análise:

ConfigMap `kubeoptix-analyzer-1-ca` não é referenciado por `envFrom`, `volume` ou `env valueFrom` nos workloads analisados.

Recomendação:

Remover se órfão ou referenciar explicitamente em `envFrom/volumeMounts`.

Origem dos dados:

- ConfigMap/`kubeoptix-analyzer-1-ca` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-analyzer-1-ca.yaml`
- ConfigMap/`kubeoptix-analyzer-1-global-ca` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-analyzer-1-global-ca.yaml`
- ConfigMap/`kubeoptix-analyzer-1-sys-config` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-analyzer-1-sys-config.yaml`
- ConfigMap/`kubeoptix-core-ai-10-ca` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-10-ca.yaml`
- ConfigMap/`kubeoptix-core-ai-10-global-ca` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-10-global-ca.yaml`
- ConfigMap/`kubeoptix-core-ai-10-sys-config` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-10-sys-config.yaml`
- ConfigMap/`kubeoptix-core-ai-6-ca` ← `/app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-6-ca.yaml`
- ConfigMap/`kubeoptix-core-ai-6-global-ca` ← `/app/data/assessment/shiftwise-`



ai/resources/configmaps/kubeoptix-core-ai-6-global-ca.yaml

- ConfigMap/kubeoptix-core-ai-6-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-6-sys-config.yaml
- ConfigMap/kubeoptix-core-ai-7-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-7-ca.yaml
- ConfigMap/kubeoptix-core-ai-7-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-7-global-ca.yaml
- ConfigMap/kubeoptix-core-ai-7-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-7-sys-config.yaml
- ConfigMap/kubeoptix-core-ai-8-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-8-ca.yaml
- ConfigMap/kubeoptix-core-ai-8-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-8-global-ca.yaml
- ConfigMap/kubeoptix-core-ai-8-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-8-sys-config.yaml
- ConfigMap/kubeoptix-core-ai-9-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-9-ca.yaml
- ConfigMap/kubeoptix-core-ai-9-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-9-global-ca.yaml
- ConfigMap/kubeoptix-core-ai-9-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-core-ai-9-sys-config.yaml
- ConfigMap/kubeoptix-dashboard-3-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-3-ca.yaml
- ConfigMap/kubeoptix-dashboard-3-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-3-global-ca.yaml
- ConfigMap/kubeoptix-dashboard-3-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-3-sys-config.yaml
- ConfigMap/kubeoptix-dashboard-4-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-4-ca.yaml
- ConfigMap/kubeoptix-dashboard-4-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-4-global-ca.yaml
- ConfigMap/kubeoptix-dashboard-4-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-4-sys-config.yaml
- ConfigMap/kubeoptix-dashboard-5-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-5-ca.yaml
- ConfigMap/kubeoptix-dashboard-5-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-5-global-ca.yaml
- ConfigMap/kubeoptix-dashboard-5-sys-config ← /app/data/assessment/shiftwise-



ai/resources/configmaps/kubeoptix-dashboard-5-sys-config.yaml

- ConfigMap/kubeoptix-dashboard-6-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-6-ca.yaml
- ConfigMap/kubeoptix-dashboard-6-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-6-global-ca.yaml
- ConfigMap/kubeoptix-dashboard-6-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-6-sys-config.yaml
- ConfigMap/kubeoptix-dashboard-7-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-7-ca.yaml
- ConfigMap/kubeoptix-dashboard-7-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-7-global-ca.yaml
- ConfigMap/kubeoptix-dashboard-7-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-dashboard-7-sys-config.yaml
- ConfigMap/kubeoptix-harvester-1-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-1-ca.yaml
- ConfigMap/kubeoptix-harvester-1-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-1-global-ca.yaml
- ConfigMap/kubeoptix-harvester-1-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-1-sys-config.yaml
- ConfigMap/kubeoptix-harvester-2-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-2-ca.yaml
- ConfigMap/kubeoptix-harvester-2-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-2-global-ca.yaml
- ConfigMap/kubeoptix-harvester-2-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-harvester-2-sys-config.yaml
- ConfigMap/kubeoptix-reporter-1-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-reporter-1-ca.yaml
- ConfigMap/kubeoptix-reporter-1-global-ca ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-reporter-1-global-ca.yaml
- ConfigMap/kubeoptix-reporter-1-sys-config ← /app/data/assessment/shiftwise-ai/resources/configmaps/kubeoptix-reporter-1-sys-config.yaml

Workloads consolidados (todos os controllers)

Controller	Quantidade
StatefulSet	5



Detalhamento por workload

Workload	Controller	Replicas (desired/ready)	Pods	QoS	CPU requests/pod	CPU limits/pod	Memory requests/pod	Memory limits/pod	Autoscaling	PD B	Probes	Strategy/Schema	ConfigMaps	Secrets	Namespace	Affinity
kubepitx-analyzer	StatefulSet	1/1	1	Burstable	250m	1000m	512Mi	2048Mi	—	—	None	RollingUpdate	—	lsm	—	None
kubepitx-core-ai	StatefulSet	1/1	1	Burstable	500m	2000m	1024Mi	4096Mi	—	—	api: readiness, liveness, startup	RollingUpdate	—	—	—	None
kubepitx-dashboards	StatefulSet	1/1	1	Burstable	50m	500m	64Mi	256Mi	—	—	dashboard: readiness, liveness	RollingUpdate	kubepitx-dashboards-env	—	—	None



Workload	Controller	Replicas (desired/ready)	Pods	QoS	CPU request/pod	CPU limit/pod	Memory request/pod	Memory limit/pod	Autoscaling	PD B	Probes	Strategy/Schedule	ConfigMaps	Secrets	Node selector	Affinity
kubeop-tix-harvester	StatefulSet	1/1	1	Burstable	500m	2000m	1024Mi	4096Mi	—	—	kubeop-tix-harvester: readiness, liveness, startup	Rolling Update	—	—	—	None
kubeop-tix-reporter	StatefulSet	1/1	1	Burstable	500m	2000m	1024Mi	4096Mi	—	kubeop-tix-reporter, minAvailable=1	kubeop-tix-reporter: readiness, liveness, startup	Rolling Update	—	—	—	None



Placement observado

Pod	Workload	Nó	Papel do nó	QoS (pod)
kubeoptix-analyzer-0	kubeoptix-analyzer	crc	—	Burstable
kubeoptix-core-ai-0	kubeoptix-core-ai	crc	—	Burstable
kubeoptix-dashboard-0	kubeoptix-dashboard	crc	—	Burstable
kubeoptix-harvester-0	kubeoptix-harvester	crc	—	Burstable
kubeoptix-reporter-0	kubeoptix-reporter	crc	—	Burstable

Comunicação

1. Externa → OpenShift

Comunicação externa — kubeoptix-dashboard

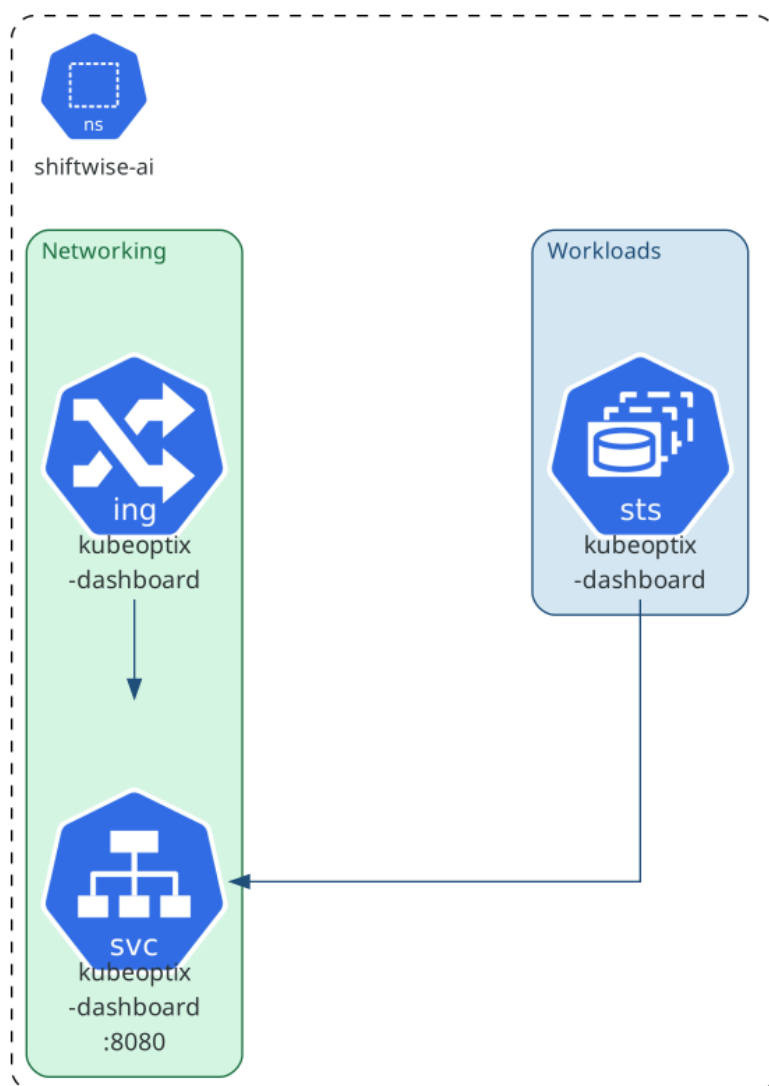
Como o tráfego externo chega ao workload via Route `kubeoptix-dashboard`?

Fato observado: 5 nó(s) e 4 relação(ões) com evidência nos artefatos YAML. Tipos de relação: ingress, router_admitted, routes_to, selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/routes.route.openshift.io/kubeoptix-dashboard.yaml`
- `resources/services/kubeoptix-dashboard.yaml`
- `resources/statefulsets.apps/kubeoptix-dashboard.yaml`



2. Comunicação interna

Comunicação interna — analyzer-api

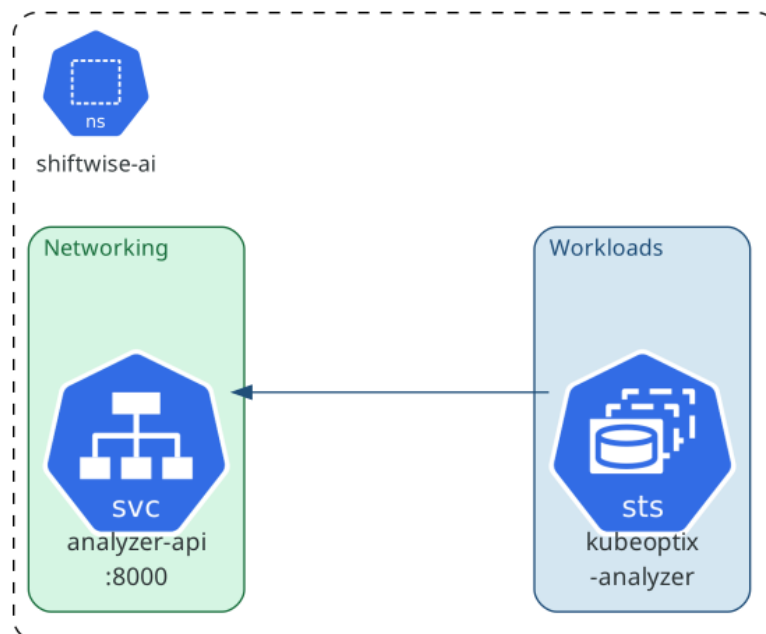
Quais pods o Service `analyzer-api` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/analyzer-api.yaml`
- `resources/statefulsets.apps/kubeoptix-analyzer.yaml`



Comunicação interna — core-ai

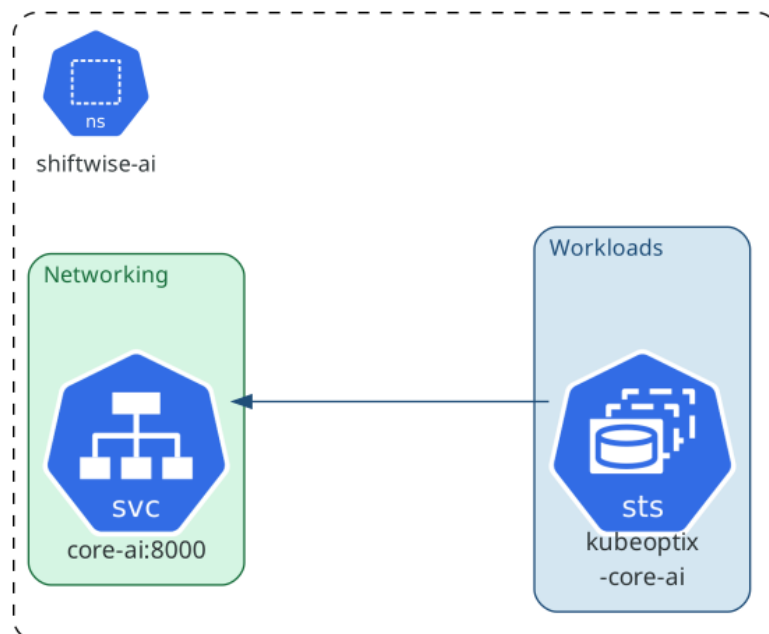
Quais pods o Service `core-ai` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/core-ai.yaml`
- `resources/statefulsets.apps/kubeoptix-core-ai.yaml`



Comunicação interna — core-ai-api

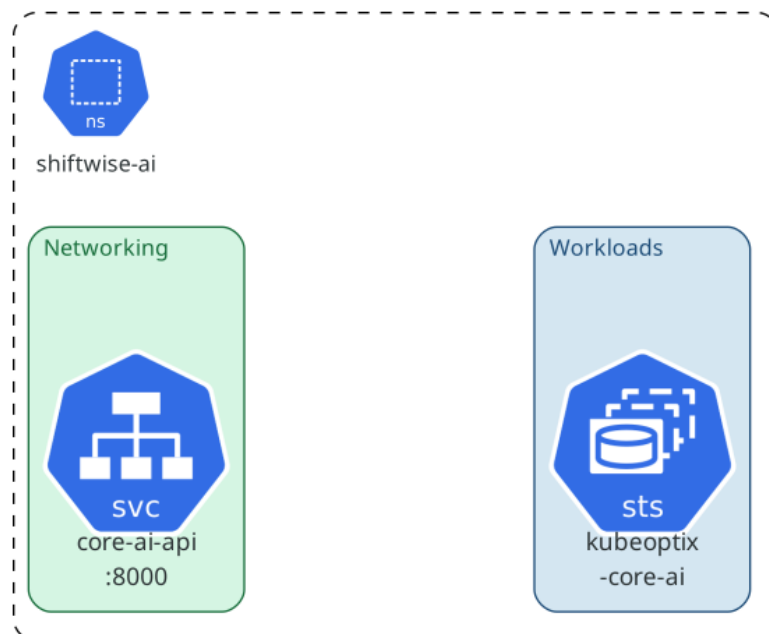
Quais pods o Service `core-ai-api` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/core-ai-api.yaml`
- `resources/statefulsets.apps/kubeoptix-core-ai.yaml`



Comunicação interna — harvester-api

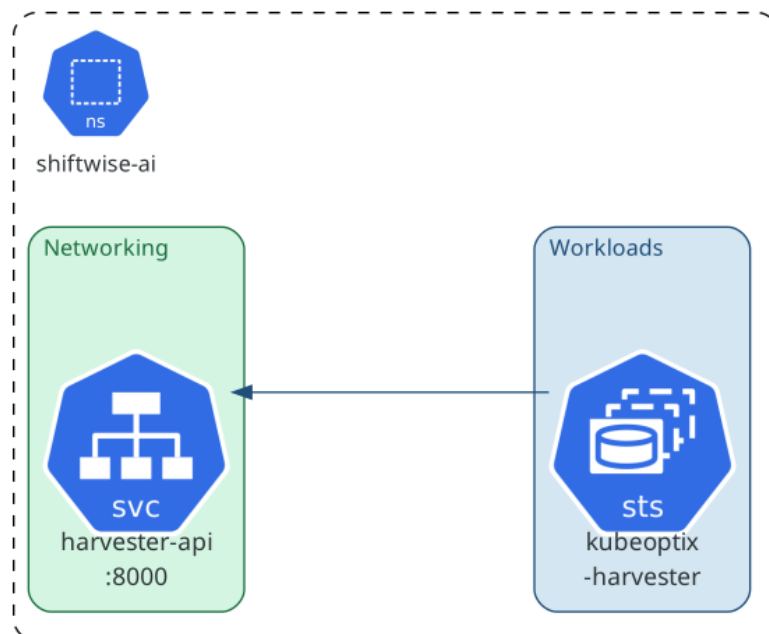
Quais pods o Service `harvester-api` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/harvester-api.yaml`
- `resources/statefulsets.apps/kubeoptix-harvester.yaml`



Comunicação interna — kubeoptix-dashboard

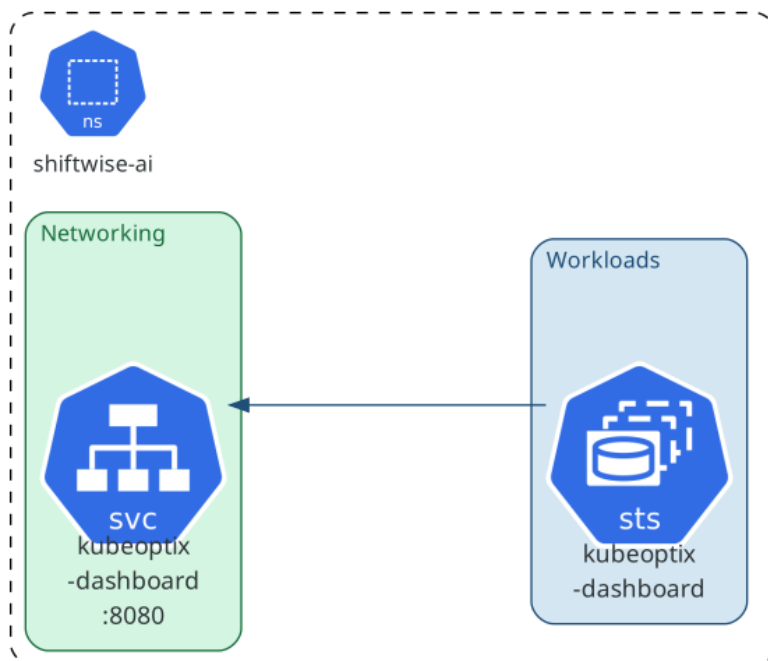
Quais pods o Service `kubeoptix-dashboard` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/kubeoptix-dashboard.yaml`
- `resources/statefulsets/apps/kubeoptix-dashboard.yaml`



Comunicação interna — reporter

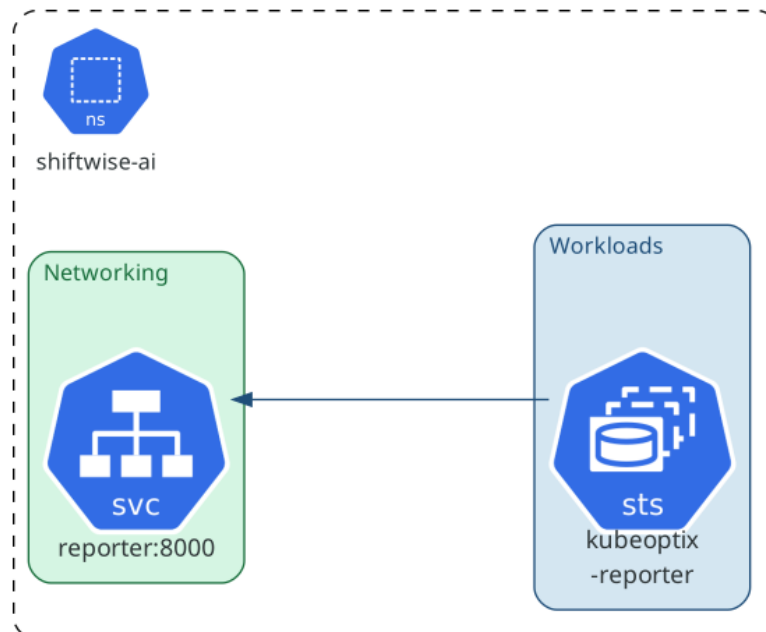
Quais pods o Service **reporter** seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo **nome** (N replicas). Services exibem a porta (**nome:8000**).

Manifests YAML utilizados:

- `resources/services/reporter.yaml`
- `resources/statefulsets/apps/kubeoptix-reporter.yaml`



Comunicação interna — reporter-api

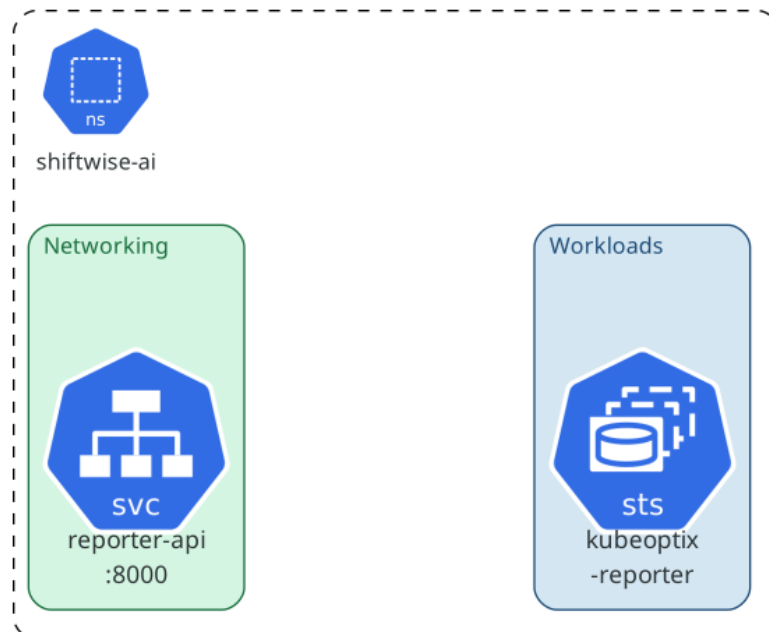
Quais pods o Service `reporter-api` seleciona no cluster?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/services/reporter-api.yaml`
- `resources/statefulsets/apps/kubeoptix-reporter.yaml`





3. Dependências externas

Dependências externas — kubeoptix-analyzer

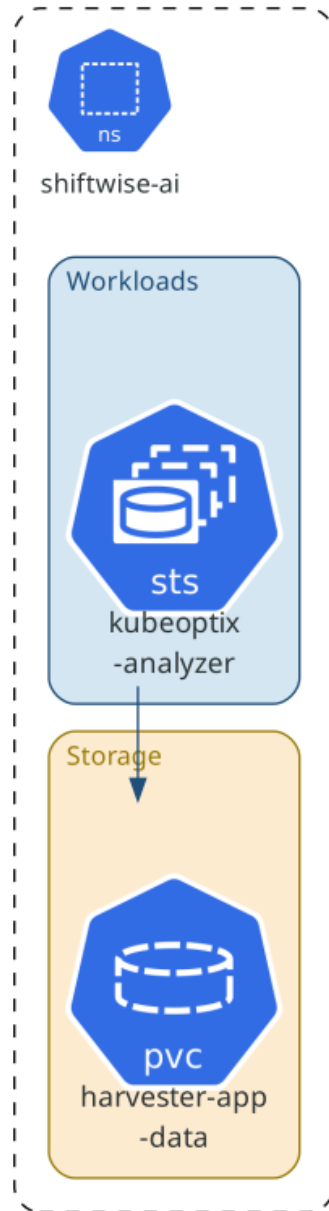
Quais sistemas externos o workload `kubeoptix-analyzer` utiliza?

Fato observado: 3 nó(s) e 2 relação(ões) com evidência nos artefatos YAML. Tipos de relação: `config_injection`, `image_pull`. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-analyzer.yaml`
- `resources/persistentvolumeclaims/harvester-app-data.yaml`





Dependências externas — kubeoptix-core-ai

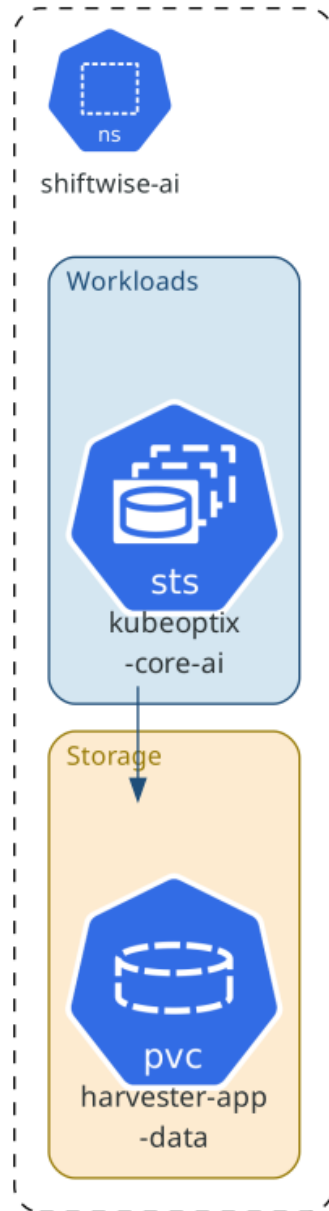
Quais sistemas externos o workload `kubeoptix-core-ai` utiliza?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: `image_pull`. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-core-ai.yaml`
- `resources/persistentvolumeclaims/harvester-app-data.yaml`





Dependências externas — kubeoptix-dashboard

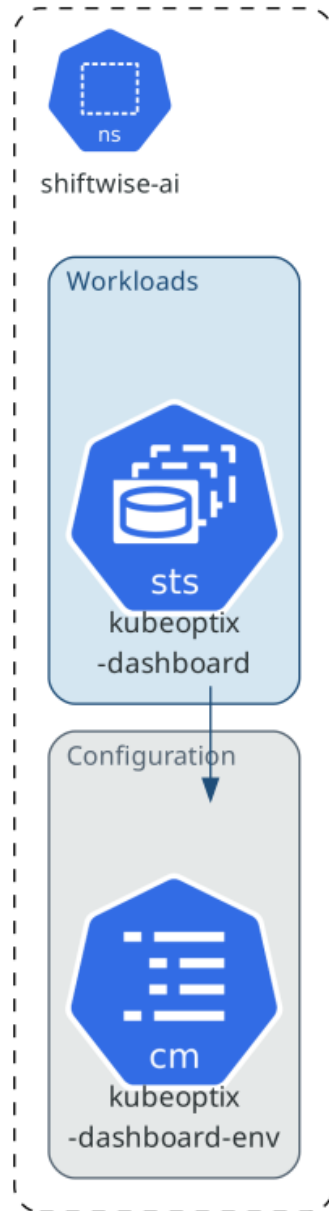
Quais sistemas externos o workload `kubeoptix-dashboard` utiliza?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: `image_pull`. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-dashboard.yaml`
- `resources/configmaps/kubeoptix-dashboard-env.yaml`





Dependências externas — kubeoptix-harvester

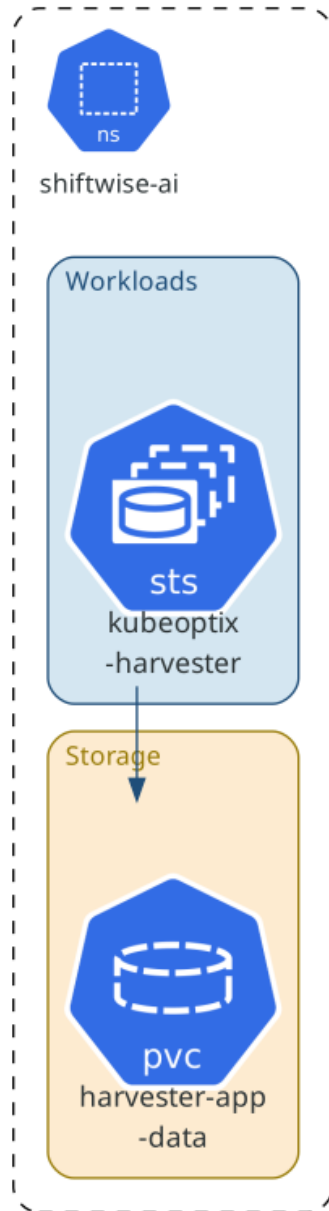
Quais sistemas externos o workload `kubeoptix-harvester` utiliza?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: `image_pull`. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome` (N replicas). Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-harvester.yaml`
- `resources/persistentvolumeclaims/harvester-app-data.yaml`





Dependências externas — kubeoptix-reporter

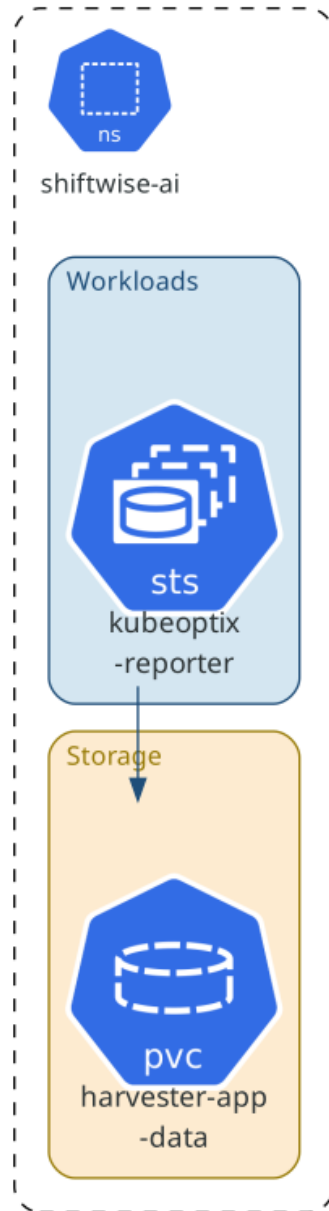
Quais sistemas externos o workload `kubeoptix-reporter` utiliza?

Fato observado: 2 nó(s) e 1 relação(ões) com evidência nos artefatos YAML. Tipos de relação: `image_pull`. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo `nome (N replicas)`. Services exibem a porta (`nome:8000`).

Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-reporter.yaml`
- `resources/persistentvolumeclaims/harvester-app-data.yaml`





Visualizações

Diagrama de workloads — kubeoptix-analyzer

Quais recursos Kubernetes cada workload utiliza ou expõe?

Fato observado: 4 nó(s) e 3 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects, uses. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com [KubeDiagrams](#) a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).

Manifests YAML utilizados:

- `resources/statefulsets/apps/kubeoptix-analyzer.yaml`
- `resources/persistentvolumeclaims/harvester-app-data.yaml`
- `resources/services/analyzer-api.yaml`

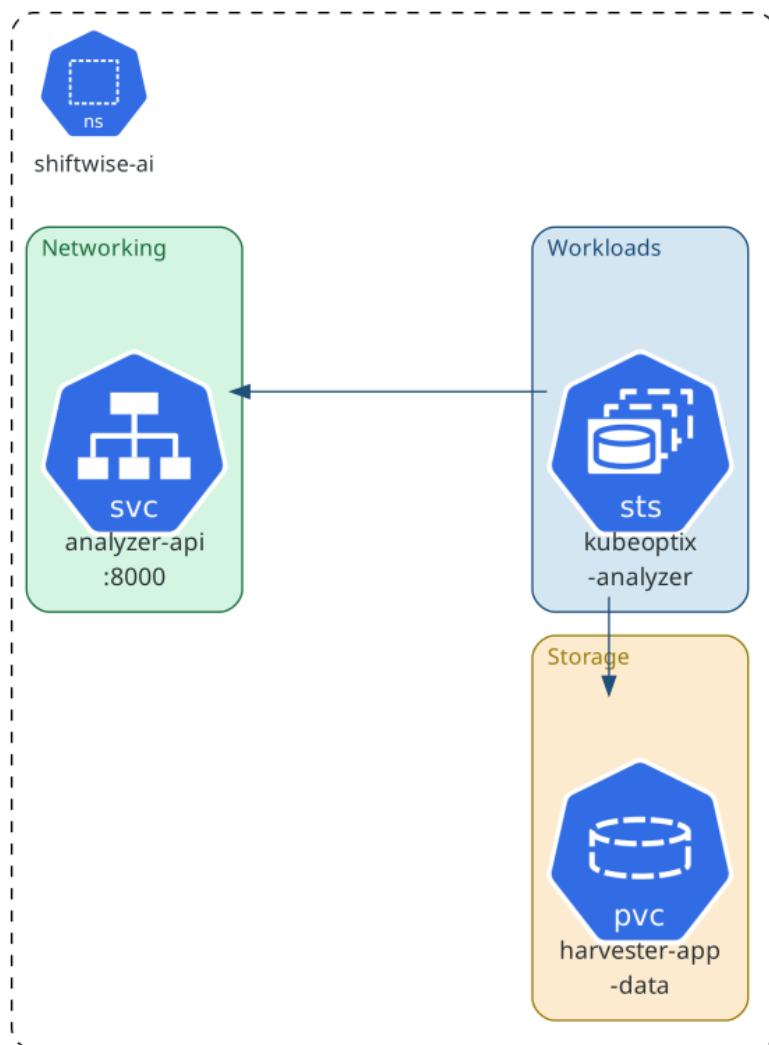


Diagrama de workloads — kubeoptix-core-ai

Quais recursos Kubernetes cada workload utiliza ou expõe?

Fato observado: 4 nó(s) e 3 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects,



uses. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).

Manifests YAML utilizados:

- resources/statefulsets.apps/kubeoptix-core-ai.yaml
- resources/persistentvolumeclaims/harvester-app-data.yaml
- resources/services/core-ai.yaml
- resources/services/core-ai-api.yaml

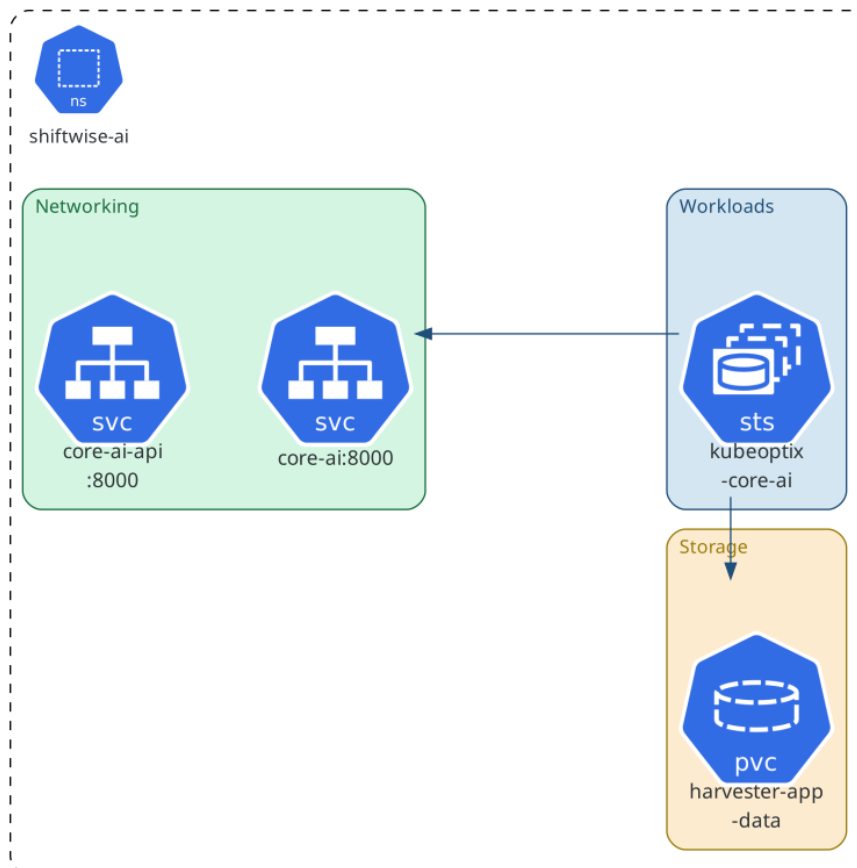


Diagrama de workloads — kubeoptix-dashboard

Quais recursos Kubernetes cada workload utiliza ou expõe?

Fato observado: 3 nó(s) e 2 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects, uses. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com KubeDiagrams a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuração (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuração → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).



Manifests YAML utilizados:

- `resources/statefulsets.apps/kubeoptix-dashboard.yaml`
- `resources/configmaps/kubeoptix-dashboard-env.yaml`
- `resources/services/kubeoptix-dashboard.yaml`

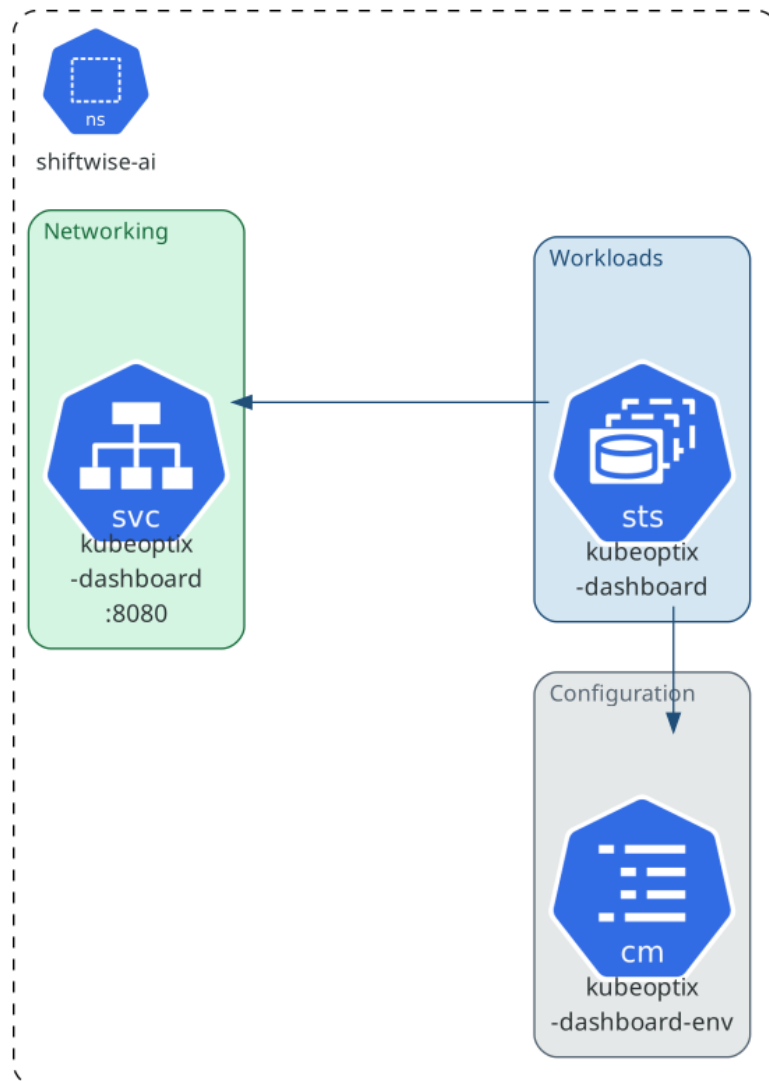


Diagrama de workloads — kubeoptix-harvester

Quais recursos Kubernetes cada workload utiliza ou expõe?



Fato observado: 3 nó(s) e 2 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects, uses. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).

Manifests YAML utilizados:

- resources/statefulsets.apps/kubeoptix-harvester.yaml
- resources/persistentvolumeclaims/harvester-app-data.yaml
- resources/services/harvester-api.yaml

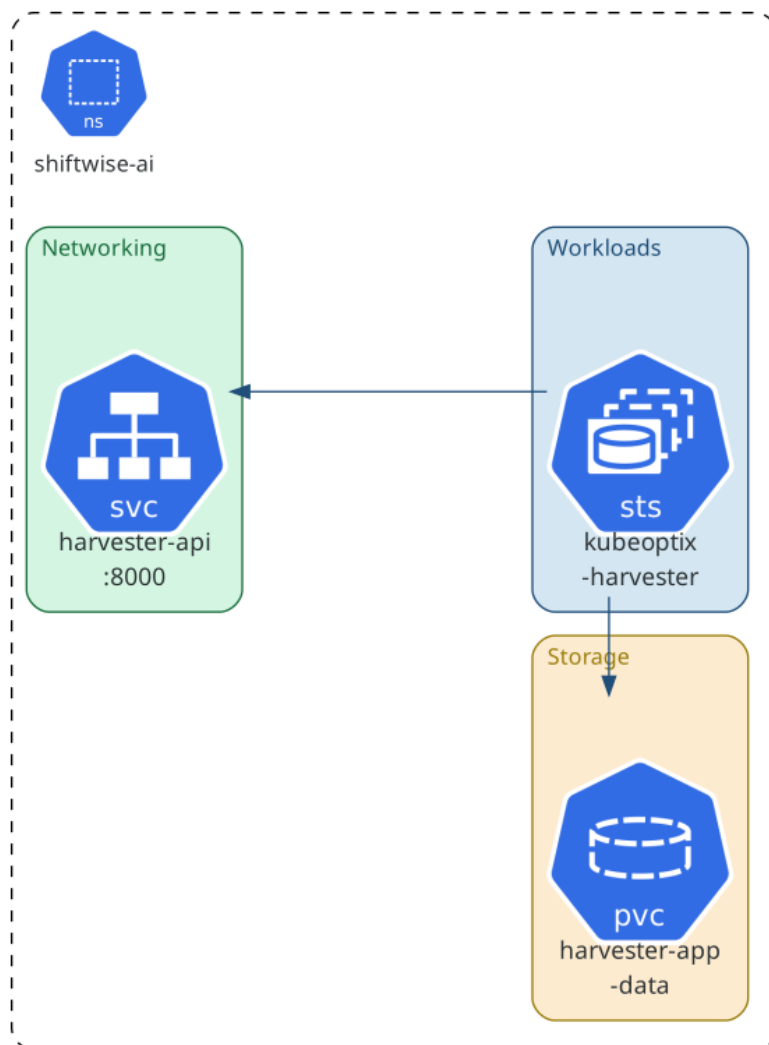


Diagrama de workloads — kubeoptix-reporter

Quais recursos Kubernetes cada workload utiliza ou expõe?

Fato observado: 4 nó(s) e 3 relação(ões) com evidência nos artefatos YAML. Tipos de relação: selects,

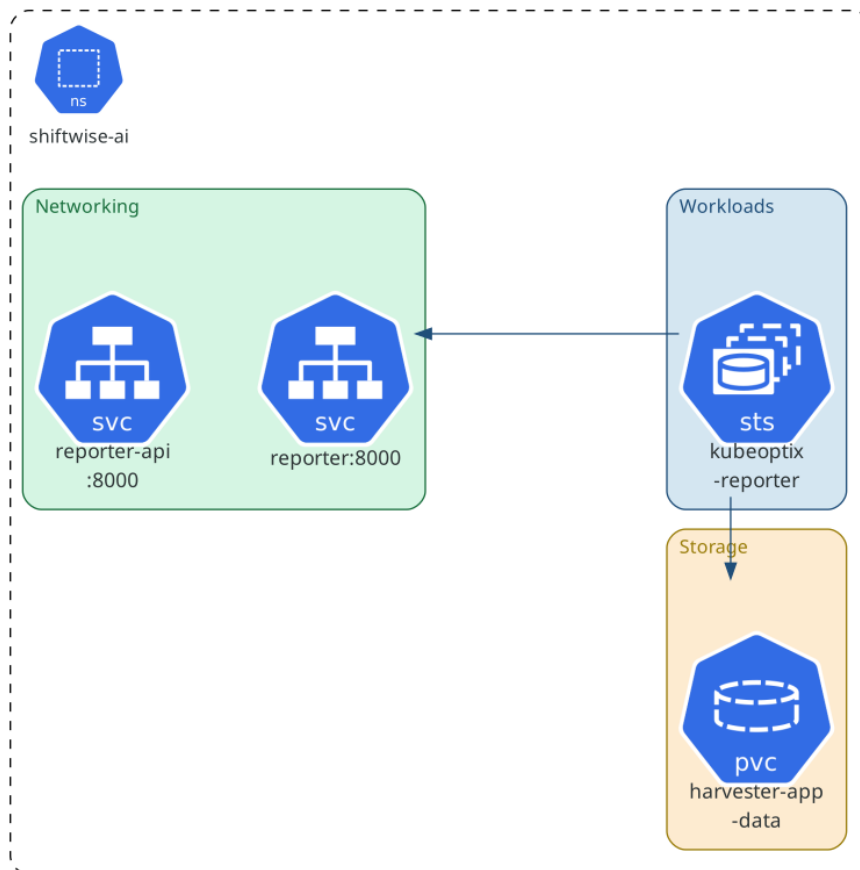


uses. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).

Manifests YAML utilizados:

- resources/statefulsets.apps/kubeoptix-reporter.yaml
- resources/persistentvolumeclaims/harvester-app-data.yaml
- resources/services/reporter.yaml
- resources/services/reporter-api.yaml



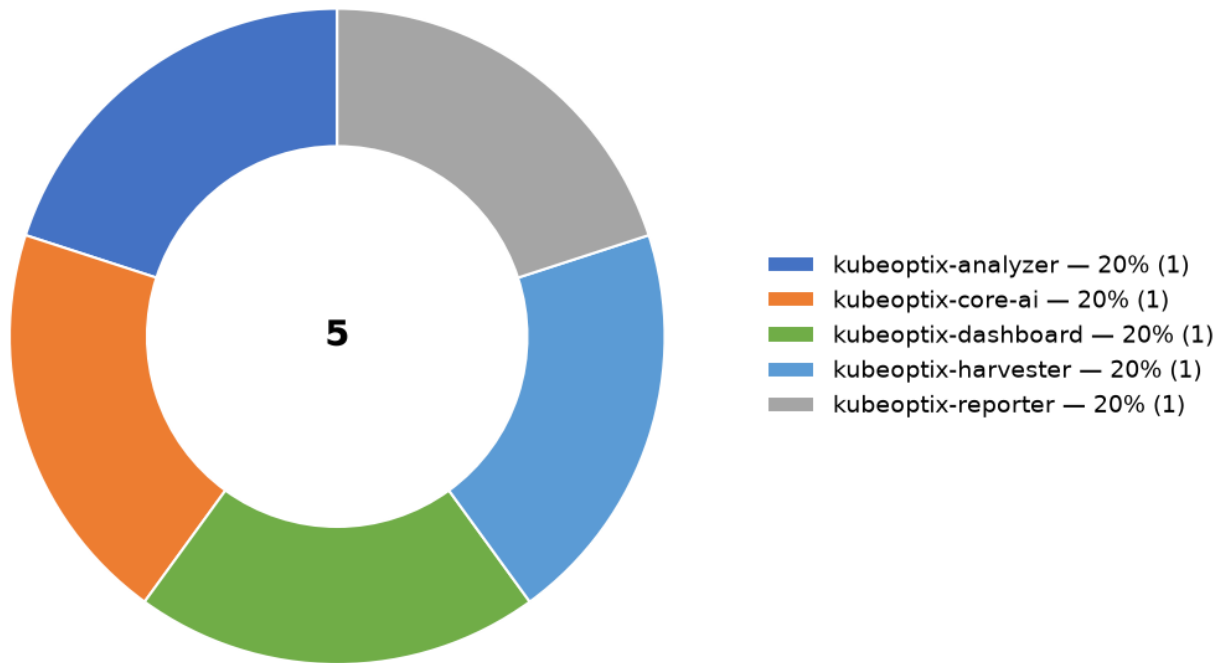
Workloads por app group

Como os workloads se distribuem por grupo de aplicação?

Valor calculado: distribuição entre 5 unidade(s) — kubeoptix-analyzer: 1, kubeoptix-core-ai: 1, kubeoptix-dashboard: 1, kubeoptix-harvester: 1, kubeoptix-reporter: 1. Contagens derivadas dos dados ingeridos, sem inferência adicional.



Workloads por app group





7. Análise de CPU

Configuração (request/limit)

Valores de **request** e **limit** abaixo são configurados nos controllers de workload — não representam uso real.

Workload	CPU request/pod	CPU limit/pod	Réplicas
kubeoptix-analyzer	250m	1000m	1
kubeoptix-core-ai	500m	2000m	1
kubeoptix-dashboard	50m	500m	1
kubeoptix-harvester	500m	2000m	1
kubeoptix-reporter	500m	2000m	1

Uso real (PodMetrics)

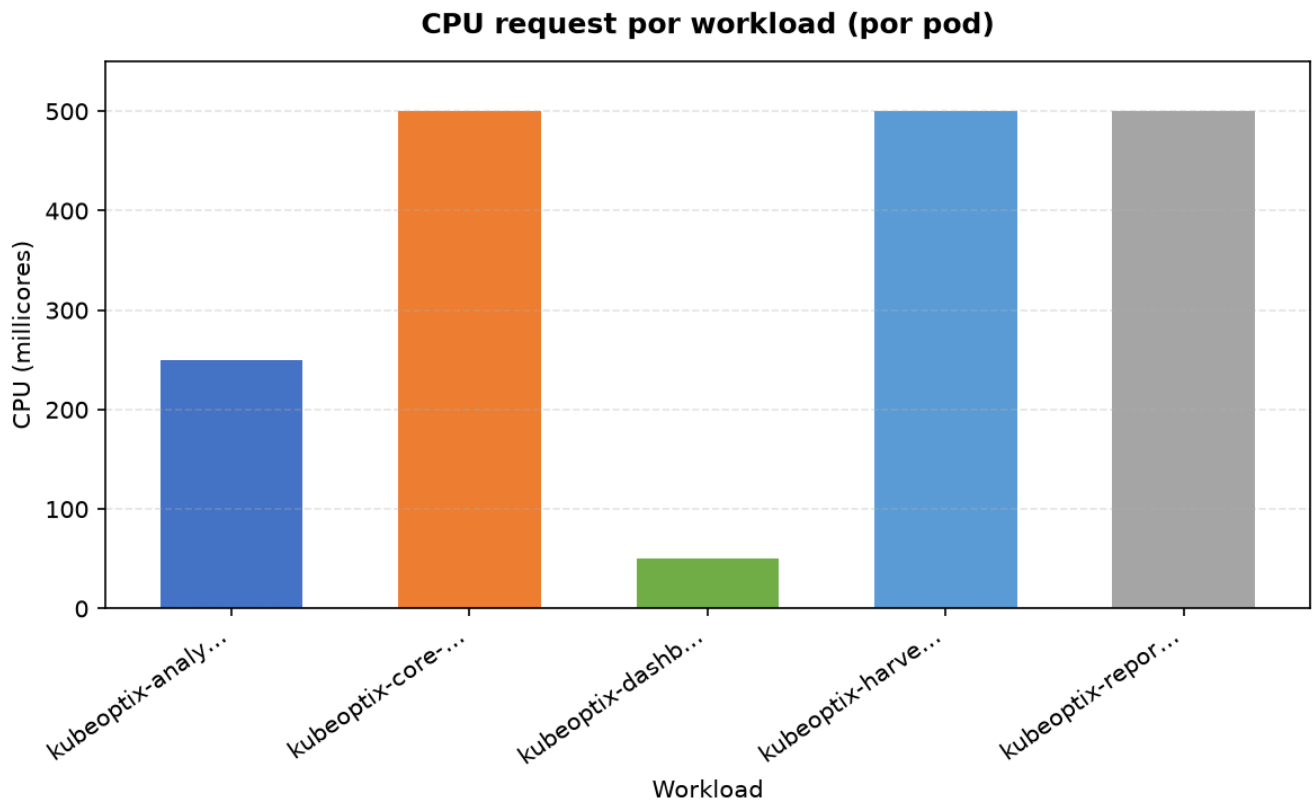
Métricas de runtime (CPU/memory **usage**) não disponíveis nos dados coletados.

Visualizações

CPU request por workload (por pod)

Quanto de CPU cada workload reserva por pod?

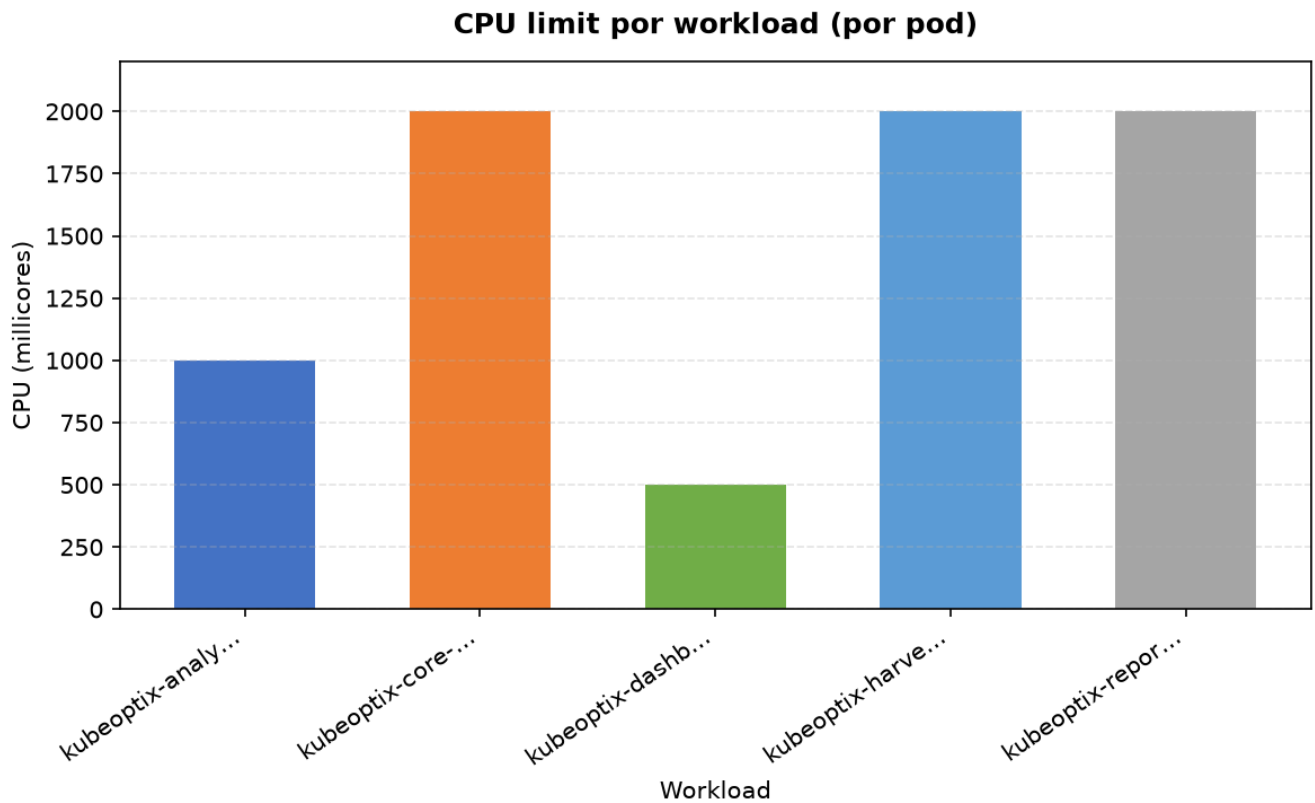
Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: CPU request. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



CPU limit por workload (por pod)

Qual o teto de CPU configurado por pod em cada workload?

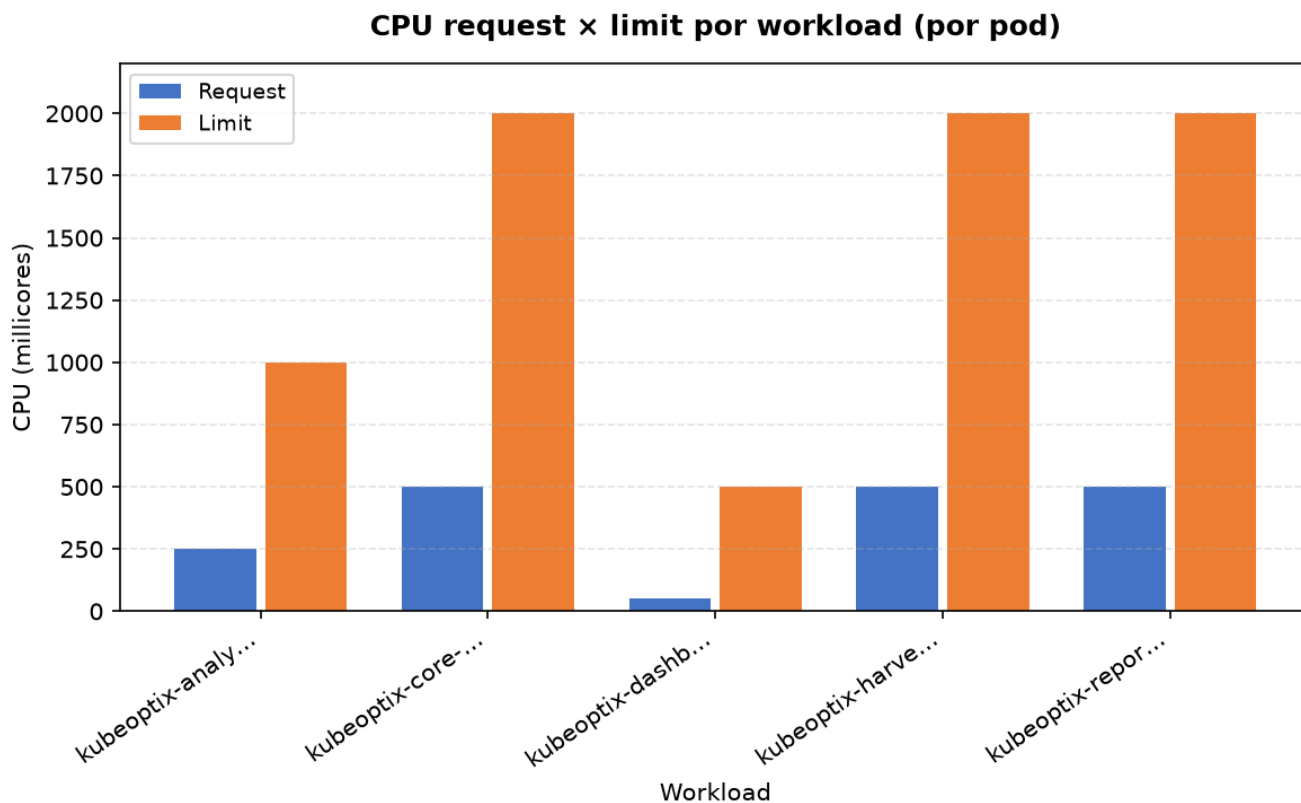
Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: CPU limit. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



CPU request × limit por workload (por pod)

Como request e limit de CPU se comparam em cada workload?

Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: Request, Limit. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



Findings de CPU e recursos relacionados:

RES-CPU-001

Categoria: CPU **Severidade:** INFO **Confiança:** HIGH

Evidências:

- Nenhuma evidência estruturada registrada.

Análise:

A análise de utilização real de CPU não foi realizada devido à ausência de métricas de runtime (PodMetrics) nos dados disponíveis.

Limitação:

Informação não disponível nos dados coletados.

8. Análise de memória

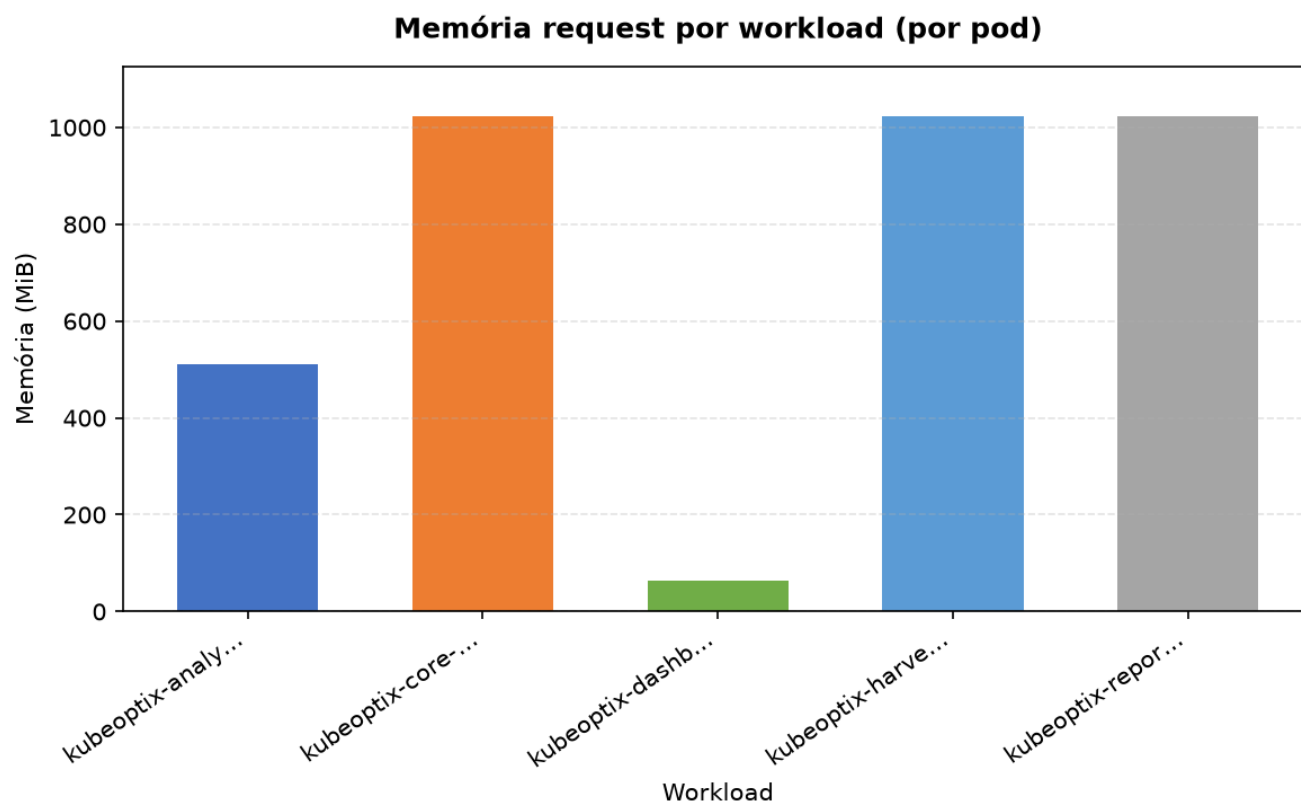
Workload	Mem request/pod	Mem limit/pod	Réplicas
kubeoptix-analyzer	512Mi	2048Mi	1
kubeoptix-core-ai	1024Mi	4096Mi	1
kubeoptix-dashboard	64Mi	256Mi	1
kubeoptix-harvester	1024Mi	4096Mi	1
kubeoptix-reporter	1024Mi	4096Mi	1

Visualizações

Memória request por workload (por pod)

Quanto de memória cada workload reserva por pod?

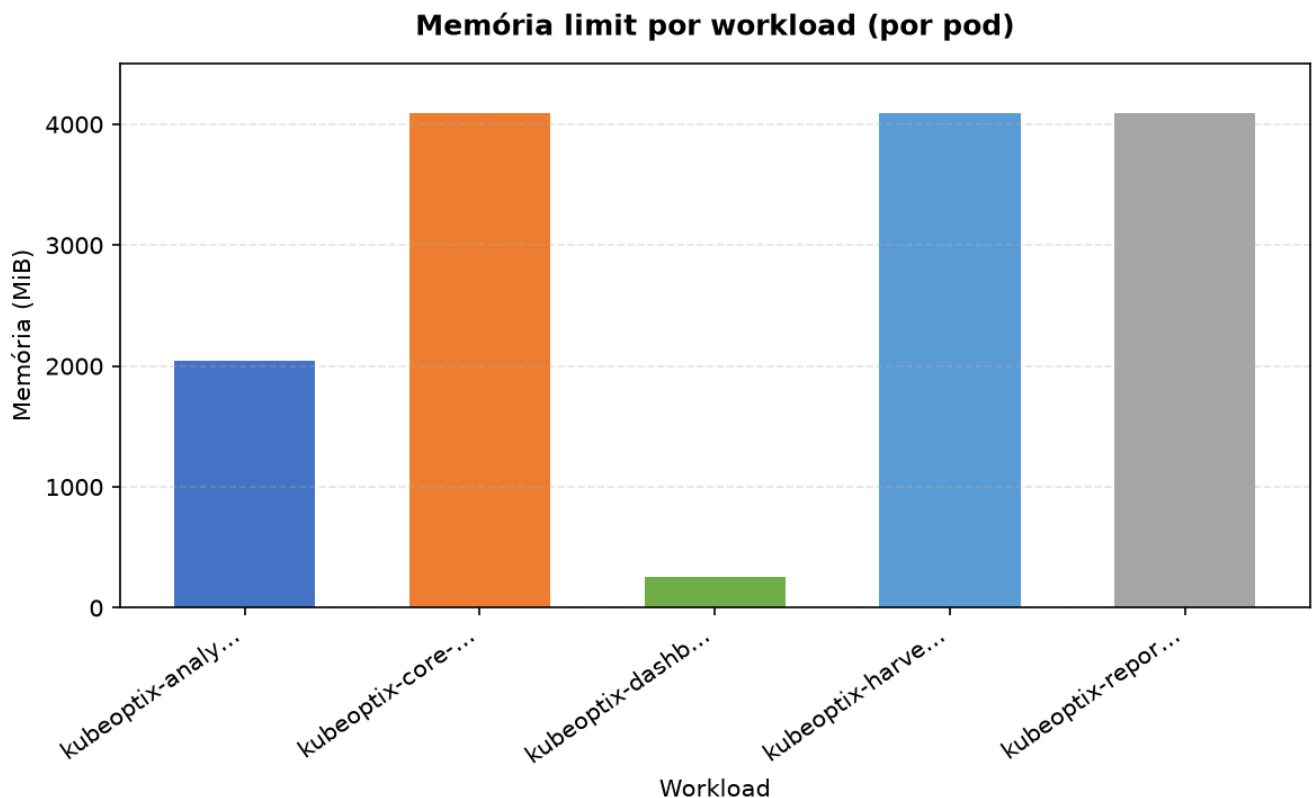
Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: Mem request. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



Memória limit por workload (por pod)

Qual o teto de memória configurado por pod em cada workload?

Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: Mem limit. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



Memória request × limit por workload (por pod)

Como request e limit de memória se comparam em cada workload?

Fato observado / valor calculado: gráfico derivado dos workloads e worknodes coletados. Séries: Request, Limit. Valores agregados do namespace são **cálculos** (soma request × réplicas). Não confundir com uso real (PodMetrics).



Findings de memória:

RES-MEM-001

Categoria: MEM **Severidade:** INFO **Confiança:** HIGH

Evidências:

- Nenhuma evidência estruturada registrada.

Análise:

A análise de utilização real de memória não foi realizada devido à ausência de métricas de runtime nos dados disponíveis.

Limitação:

Informação não disponível nos dados coletados.



9. Análise de QoS

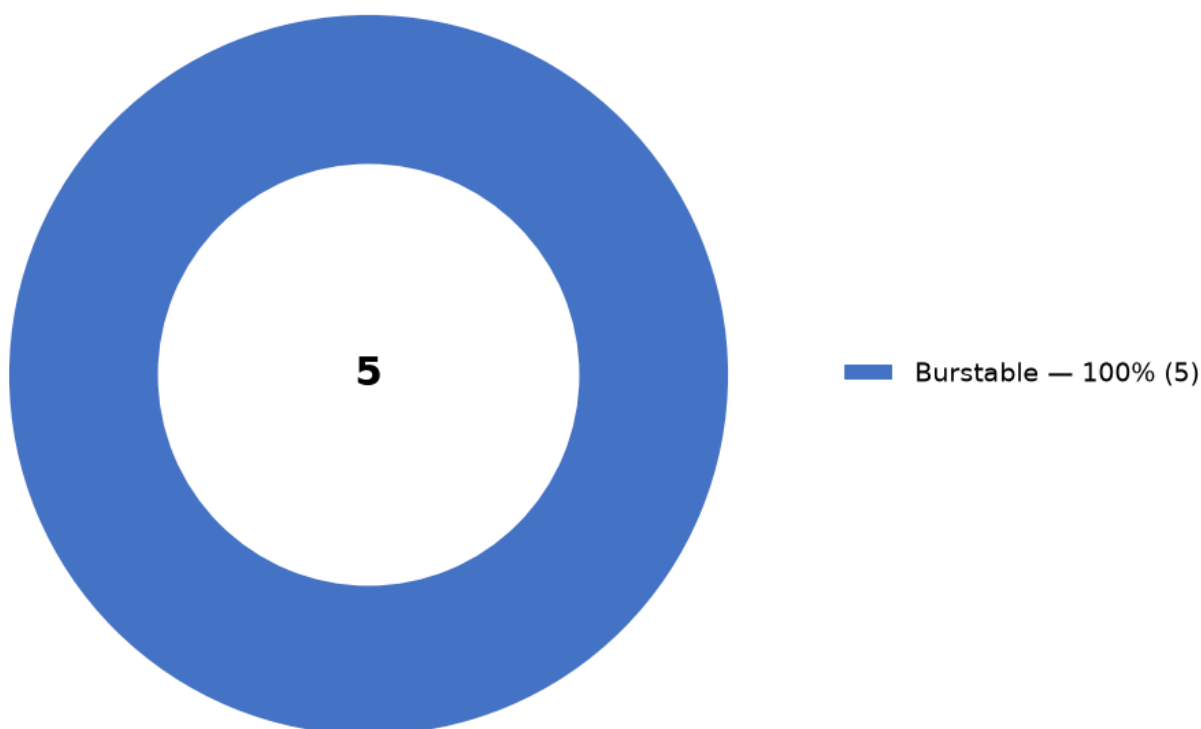
Visualizações

Distribuição de QoS

Como os workloads se distribuem entre classes QoS?

Valor calculado: distribuição entre 5 unidade(s) — Burstable: 5. Contagens derivadas dos dados ingeridos, sem inferência adicional.

Distribuição de QoS



Findings da categoria qos:

Nenhum finding nesta categoria.



10. Análise de réplicas

Findings da categoria replicas:

RES-REPLICA-001 — [REPLICA] (5 workloads; RES-REPLICA-001 ... RES-REPLICA-005)

Severidade: MEDIUM **Confiança:** HIGH **Workloads afetados (5):** kubeoptix-analyzer, kubeoptix-core-ai, kubeoptix-dashboard, kubeoptix-harvester, kubeoptix-reporter

Evidências:

Workload	Container	Evidência
kubeoptix-analyzer	—	Réplicas desejadas: 1
kubeoptix-core-ai	—	Réplicas desejadas: 1
kubeoptix-dashboard	—	Réplicas desejadas: 1
kubeoptix-harvester	—	Réplicas desejadas: 1
kubeoptix-reporter	—	Réplicas desejadas: 1

Análise:

O workload **kubeoptix-analyzer** (StatefulSet) opera com uma única réplica (spec.replicas=1).

Impacto potencial:

Risco potencial de indisponibilidade durante rolling updates ou falha de nó — não comprovado como indisponibilidade efetiva.

Recomendação:

Avaliar aumento de réplicas ou HPA para cargas que exigem HA.

Origem dos dados:

- **StatefulSet/kubeoptix-analyzer** ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml
- **StatefulSet/kubeoptix-core-ai** ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-core-ai.yaml
- **StatefulSet/kubeoptix-dashboard** ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-dashboard.yaml
- **StatefulSet/kubeoptix-harvester** ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-harvester.yaml
- **StatefulSet/kubeoptix-reporter** ← /app/data/assessment/shiftwise-



ai/resources/statefulsets.apps/kubeoptix-reporter.yaml

RES-REPLICA-006

Categoria: REPLICA **Severidade:** MEDIUM **Confiança:** MEDIUM **Workload:** kubeoptix-reporter

Evidências:

- PDB com réplica única: `minAvailable=1` ← /app/data/assessment/shiftwise-ai/resources/poddisruptionbudgets.policy/kubeoptix-reporter.yaml

Análise:

PDB `kubeoptix-reporter` com `minAvailable=1` e workload com réplica única impede evictions voluntárias durante manutenção.

Recomendação:

Aumentar réplicas ou ajustar PDB para permitir manutenção de nós.

Origem dos dados:

- `PodDisruptionBudget/kubeoptix-reporter` ← /app/data/assessment/shiftwise-ai/resources/poddisruptionbudgets.policy/kubeoptix-reporter.yaml
- `StatefulSet/kubeoptix-reporter` ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-reporter.yaml



11. Análise de probes

Findings da categoria probes:

RES-PROBE-001

Categoria: PROBE **Severidade:** MEDIUM **Confiança:** HIGH **Workload:** kubeoptix-analyzer **Container:** kubeoptix-analyzer

Evidências:

- readinessProbe ausente (spec.template.spec.containers[].readinessProbe) ←
/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml

Análise:

O container **kubeoptix-analyzer** não define readinessProbe.

Impacto potencial:

O Service pode encaminhar tráfego a pods não prontos.

Recomendação:

Adicionar readinessProbe HTTP/TCP adequada ao endpoint de saúde.

Origem dos dados:

- **StatefulSet/kubeoptix-analyzer** ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml

RES-PROBE-002

Categoria: PROBE **Severidade:** MEDIUM **Confiança:** HIGH **Workload:** kubeoptix-analyzer **Container:** kubeoptix-analyzer

Evidências:

- livenessProbe ausente (spec.template.spec.containers[].livenessProbe) ←
/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml

Análise:

O container **kubeoptix-analyzer** não define livenessProbe.

Impacto potencial:

Processos travados podem não ser reiniciados automaticamente.

Recomendação:

Adicionar livenessProbe para detectar deadlocks.

**Origem dos dados:**

- `StatefulSet/kubeoptix-analyzer` ← `/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml`

RES-PROBE-003 — [PROBE] (2 workloads; RES-PROBE-003 ... RES-PROBE-004)

Severidade: INFO **Confiança:** HIGH **Workloads afetados (2):** `kubeoptix-analyzer`, `kubeoptix-dashboard`

Evidências:

Workload	Container	Evidência
<code>kubeoptix-analyzer</code>	<code>kubeoptix-analyzer</code>	startupProbe ausente (container <code>kubeoptix-analyzer</code>)
<code>kubeoptix-dashboard</code>	<code>dashboard</code>	startupProbe ausente (container <code>dashboard</code>)

Análise:

O container `kubeoptix-analyzer` não define startupProbe.

Impacto potencial:

Aplicações com inicialização lenta podem falhar liveness prematuramente se configurada.

Recomendação:

Considerar startupProbe para apps com boot longo.

Origem dos dados:

- `StatefulSet/kubeoptix-analyzer` ← `/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml`
- `StatefulSet/kubeoptix-dashboard` ← `/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-dashboard.yaml`



12. Análise de scheduling

Findings da categoria scheduling:

Nenhum finding nesta categoria.



13. Análise de storage

Findings da categoria storage:

RES-STORAGE-001 — [STORAGE] (6 workloads; RES-STORAGE-001 ... RES-STORAGE-006)

Severidade: INFO **Confiança:** HIGH **Workloads afetados (6):** kubeoptix-analyzer, kubeoptix-core-ai, kubeoptix-harvester, kubeoptix-reporter

Evidências:

Workload	Container	Evidência
kubeoptix-analyzer	kubeoptix-analyzer	volumeMount configurado: app-data → /app/data (container kubeoptix-analyzer)
kubeoptix-core-ai	api	volumeMount configurado: tmp → /tmp (container api)
kubeoptix-core-ai	api	volumeMount configurado: logs → /app/logs (container api)
kubeoptix-core-ai	api	volumeMount configurado: data → /app/data (container api)
kubeoptix-harvester	kubeoptix-harvester	volumeMount configurado: app-data → /app/data (container kubeoptix-harvester)
kubeoptix-reporter	kubeoptix-reporter	volumeMount configurado: app-data → /app/data (container kubeoptix-reporter)

Análise:

Container kubeoptix-analyzer monta volume app-data em /app/data.

Origem dos dados:

- StatefulSet/kubeoptix-analyzer ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-analyzer.yaml
- StatefulSet/kubeoptix-core-ai ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-core-ai.yaml
- StatefulSet/kubeoptix-harvester ← /app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-harvester.yaml



- `StatefulSet/kubeoptix-reporter` ← `/app/data/assessment/shiftwise-ai/resources/statefulsets.apps/kubeoptix-reporter.yaml`

RES-STORAGE-007

Categoria: STORAGE **Severidade:** INFO **Confiança:** HIGH

Evidências:

- PVC no namespace: `harvester-app-data (10Gi)` ← `/app/data/assessment/shiftwise-ai/resources/persistentvolumeclaims/harvester-app-data.yaml`

Análise:

PVC `harvester-app-data` encontrado com request de storage `10Gi`.

Origem dos dados:

- `PersistentVolumeClaim/harvester-app-data` ← `/app/data/assessment/shiftwise-ai/resources/persistentvolumeclaims/harvester-app-data.yaml`

14. Correlação workload × worknode

Pod	Workload	Nó	Papel do nó	QoS (pod)
kubeoptix-analyzer-0	kubeoptix-analyzer	crc	—	Burstable
kubeoptix-core-ai-0	kubeoptix-core-ai	crc	—	Burstable
kubeoptix-dashboard-0	kubeoptix-dashboard	crc	—	Burstable
kubeoptix-harvester-0	kubeoptix-harvester	crc	—	Burstable
kubeoptix-reporter-0	kubeoptix-reporter	crc	—	Burstable

Visualizações

Placement workload × worknode

Em quais nós os pods de cada workload estão alocados?

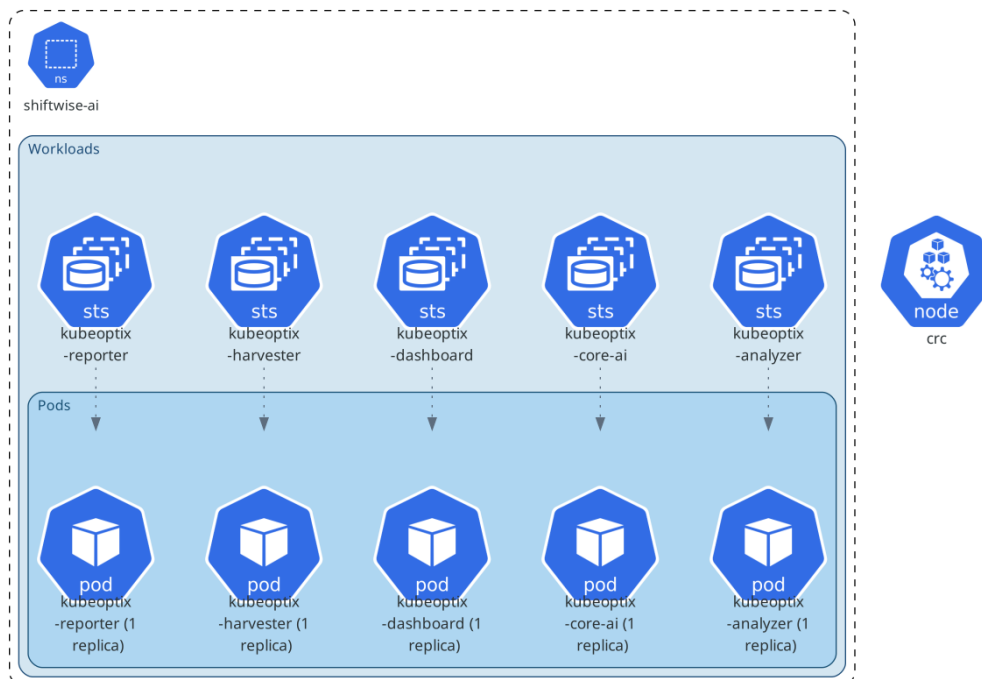
Fato observado: 6 nó(s) e 5 relação(ões) com evidência nos artefatos YAML. Tipos de relação: placed_on. Arestas sem evidência não são exibidas. Comunicação Pod → Pod não inferida.

Diagrama gerado com *KubeDiagrams* a partir dos manifests YAML listados abaixo. Cores: Workloads (azul), Pods (azul-claro), Configuration (cinza), Storage (âmbar), Networking (verde). Leitura: Workloads → Configuration → Storage à esquerda; Networking à direita. Pods equivalentes são agrupados com o rótulo nome (N replicas). Services exibem a porta (nome:8000).

Manifests YAML utilizados:

- resources/statefulsets.apps/kubeoptix-analyzer.yaml
- crc.yaml
- resources/pods/kubeoptix-analyzer-0.yaml
- resources/statefulsets.apps/kubeoptix-core-ai.yaml
- resources/pods/kubeoptix-core-ai-0.yaml
- resources/statefulsets.apps/kubeoptix-dashboard.yaml
- resources/pods/kubeoptix-dashboard-0.yaml
- resources/statefulsets.apps/kubeoptix-harvester.yaml
- resources/pods/kubeoptix-harvester-0.yaml
- resources/statefulsets.apps/kubeoptix-reporter.yaml

- ... e mais 1 arquivo(s)



Findings de correlação workload × worknode:

RES-WNODE-001

Categoria: WNODE **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- Múltiplos workloads no mesmo nó: nó=crc, workloads=['kubeoptix-analyzer', 'kubeoptix-core-ai', 'kubeoptix-dashboard', 'kubeoptix-harvester', 'kubeoptix-reporter']

Análise:

O nó **crc** hospeda pods de 5 workloads: kubeoptix-analyzer, kubeoptix-core-ai, kubeoptix-dashboard, kubeoptix-harvester, kubeoptix-reporter.

Impacto potencial:

Concentração de carga de múltiplas aplicações no mesmo nó.



RES-WNODE-002

Categoria: WNODE **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- CPU request namespace vs pool allocatable: namespace=1800m, pool=7800m
- Memory request namespace vs pool allocatable: namespace=3648Mi, pool=23571.82Mi
- Pool de scheduling: selector=todos os nós coletados, nós=1

Análise:

Requests do namespace representam 23.08% CPU e 15.48% memória do pool allocatable (1 nós, pool todos os nós coletados).

Limitação:

Capacidade reservada (requests), não consumo real. Pool compartilhado entre namespaces.

15. Anomalias identificadas

Sinais da camada estatística/ML local. Outlier estatístico **não implica** defeito operacional.

Findings estatísticos / ML:

ML-MLANOM-001

Categoria: MLANOM **Severidade:** LOW **Confiança:** MEDIUM **Workload:** kubeoptix-dashboard

Evidências:

- Isolation Forest: `decision_function=-0.001`, `contamination=0.2`
- Features analisadas: `cpu_request_m`, `mem_request_mib`, `cpu_limit_m`, `mem_limit_mib`, `replicas`, `container_count`, `cpu_limit_request_ratio`, `mem_limit_request_ratio`, `readiness_probe_ratio`, `liveness_probe_ratio`, `node_selector_flag`, `anti_affinity_flag`, `cpu_usage_request_ratio`, `mem_usage_request_ratio`

Análise:

O workload `kubeoptix-dashboard` foi classificado como anomalia multivariada pelo Isolation Forest no perfil de recursos e configuração do namespace (score -0.001).

Impacto potencial:

Combinação atípica de CPU/memória/réplicas/probes em relação aos demais workloads.

Recomendação:

Validar manualmente se a configuração é intencional antes de alterar requests ou limits.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLCLUST-001

Categoria: MLCLUST **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- K-Means: `k=2`, `seed=42`
- Composição dos grupos: `[kubeoptix-analyzer, kubeoptix-dashboard]`; `[kubeoptix-core-ai, kubeoptix-harvester, kubeoptix-reporter]`

Análise:

K-Means (k=2) agrupou os 5 workloads do namespace em perfis numéricos: `[kubeoptix-analyzer, kubeoptix-dashboard]`; `[kubeoptix-core-ai, kubeoptix-harvester, kubeoptix-reporter]`.

**Impacto potencial:**

Facilita comparar right-sizing entre workloads do mesmo grupo em vez de tratar o namespace como homogêneo.

Recomendação:

Revisar discrepâncias de recursos entre workloads dentro do mesmo grupo identificado.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLCOMP-001

Categoria: MLCOMP **Severidade:** INFO **Confiança:** MEDIUM **Workload:** kubeoptix-dashboard

Evidências:

- Distância euclidiana ao centroide (features padronizadas): 3.97 (média do namespace: 2.48)

Análise:

O workload **kubeoptix-dashboard** é o mais distante do perfil médio do namespace no espaço de features estruturadas (distância 3.97 vs média 2.48).

Impacto potencial:

Pode representar papel operacional diferente (ex.: batch vs API) ou configuração inconsistente com pares do namespace.

Recomendação:

Revisar se o perfil de recursos e réplicas está alinhado à função do workload.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLCOMP-002

Categoria: MLCOMP **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- Par de workloads: **kubeoptix-core-ai** ↔ **kubeoptix-dashboard**
- Distância euclidiana: 5.50

Análise:

Os workloads **kubeoptix-core-ai** e **kubeoptix-dashboard** são o par mais divergente do namespace



(distância 5.50 no espaço de features padronizadas).

Recomendação:

Útil para priorizar revisões de right-sizing entre componentes de arquiteturas distintas no mesmo namespace.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLSIM-001

Categoria: MLSIM **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- Par de workloads: `kubeoptix-core-ai` ↔ `kubeoptix-harvester`
- Similaridade de cosseno: 1.000 (limiar 0.9)

Análise:

Os workloads `kubeoptix-core-ai` e `kubeoptix-harvester` apresentam perfil de features estruturadas muito similar (cosseno 100.0%) no namespace.

Impacto potencial:

Candidatos naturais para comparação de utilização real e harmonização de requests/limits.

Recomendação:

Se um dos workloads apresentar métricas de runtime divergentes, investigar diferenças de código, tráfego ou configuração não capturada nas features numéricas.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLSIM-002

Categoria: MLSIM **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- Par de workloads: `kubeoptix-core-ai` ↔ `kubeoptix-reporter`
- Similaridade de cosseno: 1.000 (limiar 0.9)

Análise:

Os workloads `kubeoptix-core-ai` e `kubeoptix-reporter` apresentam perfil de features estruturadas muito



similar (cosseno 100.0%) no namespace.

Impacto potencial:

Candidatos naturais para comparação de utilização real e harmonização de requests/limits.

Recomendação:

Se um dos workloads apresentar métricas de runtime divergentes, investigar diferenças de código, tráfego ou configuração não capturada nas features numéricas.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.

ML-MLSIM-003

Categoria: MLSIM **Severidade:** INFO **Confiança:** MEDIUM

Evidências:

- Par de workloads: `kubeoptix-harvester` ↔ `kubeoptix-reporter`
- Similaridade de cosseno: 1.000 (limiar 0.9)

Análise:

Os workloads `kubeoptix-harvester` e `kubeoptix-reporter` apresentam perfil de features estruturadas muito similar (cosseno 100.0%) no namespace.

Impacto potencial:

Candidatos naturais para comparação de utilização real e harmonização de requests/limits.

Recomendação:

Se um dos workloads apresentar métricas de runtime divergentes, investigar diferenças de código, tráfego ou configuração não capturada nas features numéricas.

Limitação:

Inferência estatística / ML local sobre o namespace analisado. Outlier estatístico não implica defeito operacional; use como sinal para investigação complementar às regras determinísticas.



16. Findings

Total: 70 findings (19 entradas após agrupar RES-* equivalentes). Detalhes completos nas seções analíticas (7–15); índice resumido abaixo.

ID	Severidade	Categoria	Workload	Resumo
RES-PROBE-001	MEDIUM	PROBE	kubeoptix-analyzer	O container kubeoptix-analyzer não define readinessProbe.
RES-PROBE-002	MEDIUM	PROBE	kubeoptix-analyzer	O container kubeoptix-analyzer não define livenessProbe.
RES-REPLICA-001 ... RES-REPLICA-005 (5)	MEDIUM	REPLICA	5 workloads	O workload kubeoptix-analyzer (StatefulSet) opera com uma única réplica (spec.replicas=1).
RES-REPLICA-006	MEDIUM	REPLICA	kubeoptix-reporter	PDB kubeoptix-reporter com minAvailable=1 e workload com réplica única impede evictions voluntárias durante manuten...
ML-MLANOM-001	LOW	MLANOM	kubeoptix-dashboard	O workload kubeoptix-dashboard foi classificado como anomalia multivariada pelo Isolation Forest no perfil de recur...



ID	Severidade	Categoria	Workload	Resumo
RES-CONFIG-001 ... RES-CONFIG-042 (42)	INFO	CONFIG	42 workloads	ConfigMap <code>kubeoptix-analyzer-1-ca</code> não é referenciado por <code>envFrom</code> , <code>volume</code> ou <code>env valueFrom</code> nos workloads analisados.
RES-CPU-001	INFO	CPU	—	A análise de utilização real de CPU não foi realizada devido à ausência de métricas de runtime (PodMetrics) nos dados...
RES-MEM-001	INFO	MEM	—	A análise de utilização real de memória não foi realizada devido à ausência de métricas de runtime nos dados disponív...
ML-MLCLUST-001	INFO	MLCLUST	—	K-Means (k=2) agrupou os 5 workloads do namespace em perfis numéricos: [kubeoptix-analyzer, kubeoptix-dashboard]; [ku...



ID	Severidade	Categoria	Workload	Resumo
ML-MLCOMP-001	INFO	MLCOMP	kubeoptix-dashboard	O workload kubeoptix-dashboard é o mais distante do perfil médio do namespace no espaço de features estruturadas (d...
ML-MLCOMP-002	INFO	MLCOMP	—	Os workloads kubeoptix-core-ai e kubeoptix-dashboard são o par mais divergente do namespace (distância 5.50 no es...
ML-MLSIM-001	INFO	MLSIM	—	Os workloads kubeoptix-core-ai e kubeoptix-harvester apresentam perfil de features estruturadas muito similar (co...
ML-MLSIM-002	INFO	MLSIM	—	Os workloads kubeoptix-core-ai e kubeoptix-reporter apresentam perfil de features estruturadas muito similar (cos...



ID	Severidade	Categoria	Workload	Resumo
ML-MLSIM-003	INFO	MLSIM	—	Os workloads kubeoptix-harvester e kubeoptix-reporter apresentam perfil de features estruturadas muito similar (c...
RES-PROBE-003 ... RES-PROBE-004 (2)	INFO	PROBE	2 workloads	O container kubeoptix-analyzer não define startupProbe .
RES-STORAGE-001 ... RES-STORAGE-006 (6)	INFO	STORAGE	6 workloads	Container kubeoptix-analyzer monta volume app-data em /app/data .
RES-STORAGE-007	INFO	STORAGE	—	PVC harvester-app-data encontrado com request de storage 10Gi .
RES-WNODE-001	INFO	WNODE	—	O nó crc hospeda pods de 5 workloads: kubeoptix-analyzer , kubeoptix-core-ai , kubeoptix-dashboard , kubeoptix-harvest...



ID	Severidade	Categoria	Workload	Resumo
RES-WNODE-002	INFO	WNODE	—	Requests do namespace representam 23.08% CPU e 15.48% memória do pool allocatable (1 nós, pool `todos os nós coletado...

Visualizações

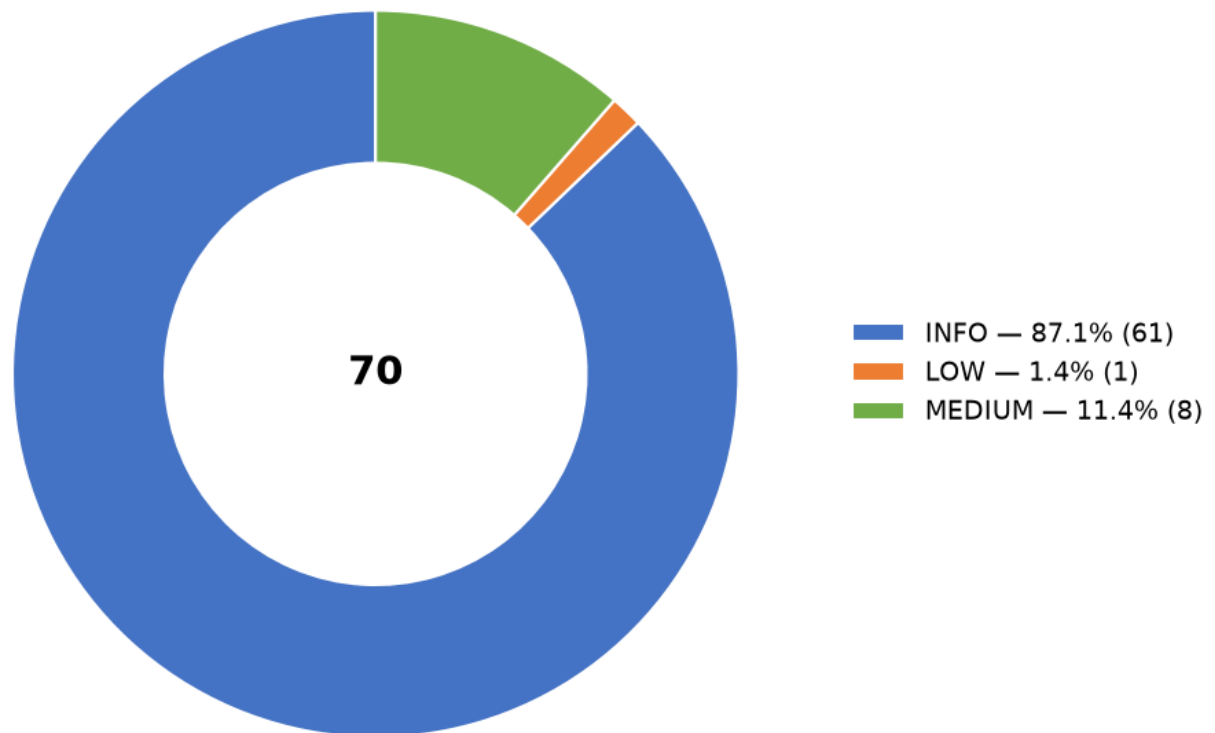
Findings por severidade

Qual a distribuição de severidade dos findings?

Valor calculado: distribuição entre 70 unidade(s) — INFO: 61, LOW: 1, MEDIUM: 8. Contagens derivadas dos dados ingeridos, sem inferência adicional.



Findings por severidade



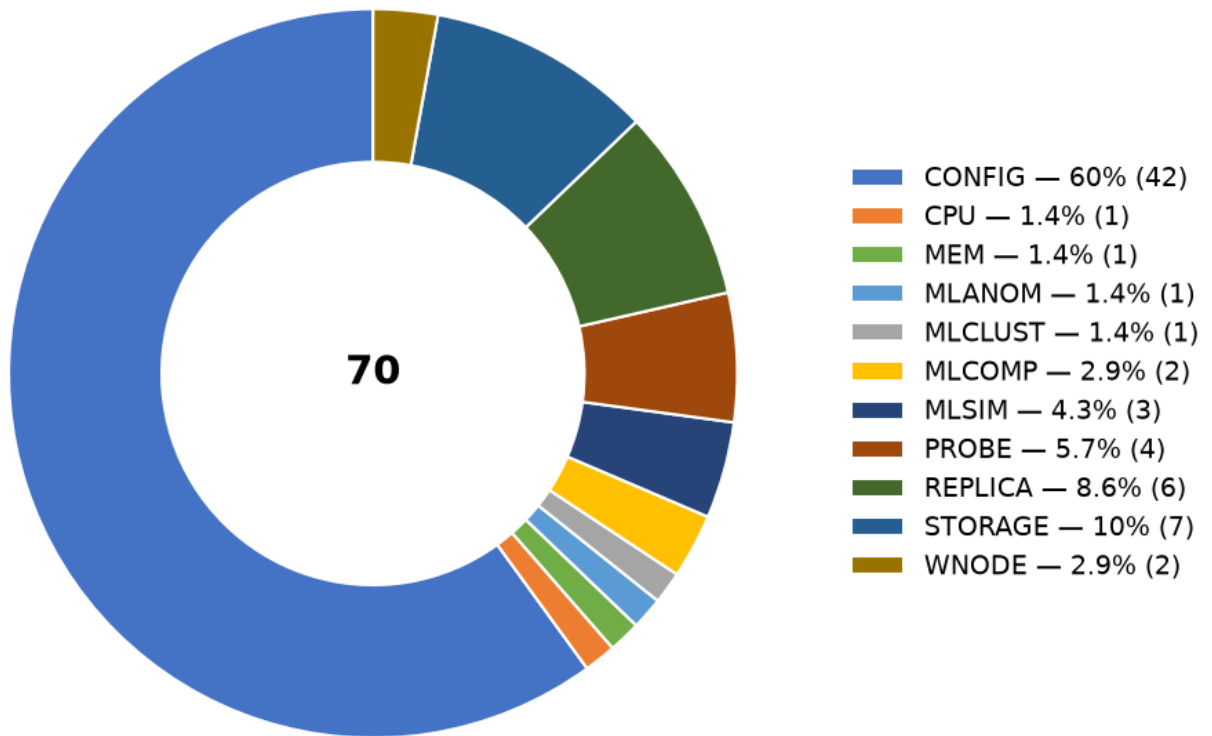
Findings por categoria

Quais categorias concentram mais findings?

Valor calculado: distribuição entre 70 unidade(s) — CONFIG: 42, CPU: 1, MEM: 1, MLANOM: 1, MLCLUST: 1, MLCOMP: 2, MLSIM: 3, PROBE: 4, REPLICA: 6, STORAGE: 7, WNODE: 2. Contagens derivadas dos dados ingeridos, sem inferência adicional.



Findings por categoria





17. Oportunidades de otimização

- Adicionar readinessProbe HTTP/TCP adequada ao endpoint de saúde. (RES-PROBE-001)
 - Adicionar livenessProbe para detectar deadlocks. (RES-PROBE-002)
 - Avaliar aumento de réplicas ou HPA para cargas que exigem HA. (RES-REPLICA-001)
 - Aumentar réplicas ou ajustar PDB para permitir manutenção de nós. (RES-REPLICA-006)
 - Remover se órfão ou referenciar explicitamente em envFrom/volumeMounts. (RES-CONFIG-001)
-



18. Riscos

Nenhum risco classificado como HIGH ou CRITICAL.



19. Recomendações

Plano de ação

Prioridade	Ação	Finding	Critério de aceite
Média	Adicionar readinessProbe HTTP/TCP adequada ao endpoint de saúde.	RES-PROBE-001	Probes respondem OK em ambiente saudável; pod reinicia em falha simulada
Média	Adicionar livenessProbe para detectar deadlocks.	RES-PROBE-002	Probes respondem OK em ambiente saudável; pod reinicia em falha simulada
Média	Avaliar aumento de réplicas ou HPA para cargas que exigem HA.	RES-REPLICA-001	Réplicas distribuídas em nós distintos (oc get pods -o wide)
Média	Aumentar réplicas ou ajustar PDB para permitir manutenção de nós.	RES-REPLICA-006	Réplicas distribuídas em nós distintos (oc get pods -o wide)
Baixa	Validar manualmente se a configuração é intencional antes de alterar requests ou limits.	ML-MLANOM-001	Investigação documentada; sem ação obrigatória sem evidência adicional
Baixa	Revisar discrepâncias de recursos entre workloads dentro do mesmo grupo identificado.	ML-MLCLUST-001	Investigação documentada; sem ação obrigatória sem evidência adicional
Baixa	Revisar se o perfil de recursos e réplicas está alinhado à função do workload.	ML-MLCOMP-001	Investigação documentada; sem ação obrigatória sem evidência adicional
Baixa	Útil para priorizar revisões de right-sizing entre componentes de arquiteturas distintas no mesmo namespace.	ML-MLCOMP-002	Investigação documentada; sem ação obrigatória sem evidência adicional



Prioridade	Ação	Finding	Critério de aceite
Baixa	Se um dos workloads apresentar métricas de runtime divergentes, investigar diferenças de código, tráfego ou configuração não capturada nas features numéricas.	ML-MLSIM-001	Investigação documentada; sem ação obrigatória sem evidência adicional
Baixa	Remover se órfão ou referenciar explicitamente em envFrom/volumeMounts.	RES-CONFIG-001	Validar em ambiente não produtivo antes de promover
Baixa	Considerar startupProbe para apps com boot longo.	RES-PROBE-003	Probes respondem OK em ambiente saudável; pod reinicia em falha simulada

Lista consolidada

- Adicionar readinessProbe HTTP/TCP adequada ao endpoint de saúde. (RES-PROBE-001)
- Adicionar livenessProbe para detectar deadlocks. (RES-PROBE-002)
- Avaliar aumento de réplicas ou HPA para cargas que exigem HA. (RES-REPLICA-001)
- Aumentar réplicas ou ajustar PDB para permitir manutenção de nós. (RES-REPLICA-006)
- Validar manualmente se a configuração é intencional antes de alterar requests ou limits. (ML-MLANOM-001)
- Remover se órfão ou referenciar explicitamente em envFrom/volumeMounts. (RES-CONFIG-001)
- Revisar discrepâncias de recursos entre workloads dentro do mesmo grupo identificado. (ML-MLCLUST-001)
- Revisar se o perfil de recursos e réplicas está alinhado à função do workload. (ML-MLCOMP-001)
- Útil para priorizar revisões de right-sizing entre componentes de arquiteturas distintas no mesmo namespace. (ML-MLCOMP-002)
- Se um dos workloads apresentar métricas de runtime divergentes, investigar diferenças de código, tráfego ou configuração não capturada nas features numéricas. (ML-MLSIM-001)
- Considerar startupProbe para apps com boot longo. (RES-PROBE-003)



20. Conclusão

O namespace **shiftwise-ai** apresenta **5 workload(s)** analisado(s). Há oportunidades de revisão de CPU request com base no uso medido. Foram registrados 0 finding(s) CRITICAL e 0 HIGH. Priorizar distribuição de réplicas e scheduling, otimização de CPU request. Validação com séries históricas de 7–30 dias é recomendada antes de alterações em produção.



21. Limitações da análise

- Métricas de runtime (PodMetrics) ausentes ou incompletas: análise de utilização real limitada.
 - Correlação workload × worknode baseada em requests configurados e snapshot de placement; não reflete consumo real nem alocação cluster-wide por nó.
 - Features de usage/request derivadas de PodMetrics ausentes; imputação por mediana da coluna ou exclusão implícita.
 - Este relatório **não** inclui conteúdo de Secrets, eventos de OOMKilled, throttling de CPU ou séries temporais — salvo quando explicitamente presentes nos artefatos ingeridos.
-



22. Referências

Fontes consultadas para fundamentar metodologia, terminologia e recomendações deste relatório.

Prioridade: documentação e publicações Red Hat; referências upstream complementam quando não há equivalente específico em OpenShift.

Documentação técnica — Red Hat

Tema	Referência
Visão geral de workloads	Red Hat OpenShift — Gerenciamento de workloads
CPU e memória (requests/limits)	Red Hat OpenShift — Recursos de computação para scheduling
QoS e overcommit	Red Hat OpenShift — Overcommit de recursos no cluster
Probes (liveness, readiness, startup)	Red Hat OpenShift — Monitoramento de saúde de aplicações
Scheduling e placement	Red Hat OpenShift — Sobre o scheduler de Pods
HPA e réplicas	Red Hat OpenShift — Autoscaling de Pods
Monitoramento e métricas	Red Hat OpenShift — Visão geral de monitoramento
Storage e PVCs	Red Hat OpenShift — Armazenamento persistente
Routes e TLS	Red Hat OpenShift — Configuração de Routes
Operadores (OLM)	Red Hat OpenShift — Operator Lifecycle Manager (OLM)
ConfigMaps	Red Hat OpenShift — ConfigMaps em aplicações

Documentação técnica complementar (upstream)

Tema	Referência
Métricas de runtime (PodMetrics)	Kubernetes — Pipeline de métricas de recursos
Referência upstream de recursos	Kubernetes — Gerenciamento de recursos de Pods e containers



Referências bibliográficas — Red Hat

Referência	Relevância
Red Hat, Inc. <i>Red Hat OpenShift Container Platform — Documentação oficial</i> . docs.openshift.com.	Referência primária para operação, networking, operadores e workloads.
Duncan, C.; Mier, T.; Brindley, A. <i>OpenShift for Developers</i> . O'Reilly Media, 2ª ed., 2021.	Desenvolvimento e operação de aplicações em OpenShift.
Ibsen, R.; Schimanski, M. <i>Kubernetes Patterns</i> . O'Reilly Media, 2ª ed., 2020.	Padrões de design para workloads em plataformas Kubernetes/OpenShift.
Red Hat, Inc. <i>OpenShift Container Platform — Scalability and Performance Guide</i> . docs.openshift.com.	Dimensionamento de recursos, réplicas e capacidade de cluster.

Referências bibliográficas complementares

Referência	Relevância
Beyer, B. et al. <i>Site Reliability Engineering: How Google Runs Production Systems</i> . O'Reilly Media, 2016.	Práticas complementares de confiabilidade e observabilidade.
Burns, B.; Grant, B.; Oppenheimer, D.; Brewer, E.; Wilkes, J. <i>Borg, Omega, and Kubernetes</i> . ACM Queue, 14(1), 2016.	Contexto histórico de orquestração em larga escala.